

1. Context

1.1. Introducció

El projecte ***Refactorització i millora de rendiment d'una aplicació empresarial del sector de l'automoció*** és un Treball de Fi de Grau (TFG) que pertany als estudis de Grau en Enginyeria Informàtica de la Facultat d'Informàtica de Barcelona (FIB), a la Universitat Politècnica de Catalunya (UPC), en l'especialització de l'Enginyeria del Software. Aquest projecte s'ha realitzat en la modalitat B, és a dir, mitjançant un conveni de cooperació educativa entre la UPC i una empresa, en aquest cas, Volkswagen Group Services (VWGS).

Volkswagen Group Services [\[1\]](#) forma part del Grup Volkswagen, un grup d'empreses multinacional alemany i un dels majors fabricants d'automòbils del món, i existeix per donar suport exclusivament a les diferents empreses del grup a escala mundial. Ofereix serveis especialitzats i interdisciplinaris en àrees de tecnologia, operacions empresarials, logística, serveis comercials i serveis administratius.

Així doncs, aquest projecte es desenvolupa dins del departament de *Commercial IT Services* que es dedica a donar suport i solucions IT als projectes relacionats amb els processos comercials.

1.2. Problema a resoldre

Dins del marc de l'estratègia tecnològica per a l'ús de tecnologies més actuals en les aplicacions de la companyia Volkswagen AG, l'empresa matriu del Grup Volkswagen a Alemanya, es crea un programa anomenat NADIN que gestiona un canvi global en l'arquitectura de tots els aplicatius dins del seu mercat. Aquesta transformació implica l'adopció de solucions tecnològiques adaptades a les noves necessitats de negoci. En aquest context, sorgeix la demanda d'una solució que actuï com a bus d'integració permetent la interoperabilitat entre sistemes, l'estandardització de processos i la incorporació àgil de les noves funcionalitats dins del mercat anglès amb NADIN. A aquesta solució se li posa el nom de NiX UK i es desenvolupa a Espanya com una capa de serveis basada en la tecnologia *cloud* que proveeix de solucions i aplicacions dirigides als importadors del grup segons les necessitats del negoci.

En un cert moment, en paral·lel, es comença a desenvolupar una solució molt semblant per al mercat de mediterrani que anomenen NiX MED.

Una sèrie d'escenaris van portar a la realització d'una auditoria externa especialitzada en Infraestructura de Clau Pública (PKI) que va revelar múltiples deficiències prou crítiques que es va determinar discontinuar el programa NADIN el 2023, NiX UK inclòs.

NiX MED, que en aquell moment estava en les fases inicials del desenvolupament, s'ha anat desenvolupant lentament fins al dia d'avui i serà el projecte en què es treballarà aquest TFG.

1.3. Definicions

Tecnologia *cloud*: Model d'emmagatzematge i gestió de recursos informàtics a través d'Internet que permet accedir, processar i allotjar dades o aplicacions de manera remota. Ofereix flexibilitat, escalabilitat i reducció de costos, ja que elimina la necessitat d'infraestructura local pròpia.

Bus d'integració: Infraestructura de programari que permet la comunicació i intercanvi de dades entre diferents aplicacions o serveis dins d'un mateix entorn informàtic. Actua com a pont centralitzat, garantint que els sistemes, tot i ser independents o heterogenis, puguin connectar-se, compartir informació i treballar de manera coordinada.

Microserveis: Model d'arquitectura de programari en què una aplicació es divideix en petits serveis independents, cadascun responsable d'una funcionalitat concreta. Aquesta estructura permet desenvolupar, desplegar i escalar cada servei de manera autònoma, facilitant el manteniment, la flexibilitat i l'evolució del sistema.

1.4. Actors implicats

En aquest apartat es llistarà els actors implicats en aquest projecte juntament amb una breu descripció dels seus rols i objectius.

Director del TFG

En Eduardo Turmo Planuch de Volkswagen Group Services és l'encarregat de supervisar la compleció dels objectius del projecte i és un dels principals interessats en l'èxit d'aquest.

Ponent del TFG

En Carles Farré Tost és el ponent del TFG, encarregat de guiar el treball i verificar la seva correctesa.

Autor del TFG

L'autor del TFG és l'encarregat del desenvolupament del projecte. És un altre dels principals interessats en completar tots els objectius.

Equip de desenvolupadors de VWGS

L'equip de desenvolupadors, juntament amb l'autor, són els encarregats de la producció del *software* del projecte. Dins d'aquest equip podem diferenciar dos rols:

- *Scrum Master*: És l'encarregat de gestionar i maximitzar el valor del producte. Actua com a nexa entre les necessitats del negoci i les capacitats tècniques de l'equip.
- Desenvolupadors: Un grup de tres persones que s'encarreguen d'implementar les funcionalitats que requereix el client.

Grup Volkswagen

Aquest projecte, tot i que té com a possibles clients totes les marques que formen part del grup Volkswagen, es centrarà en dos clients: SEAT i VGED. Aquest últim engloba les marques d'AUDI, SKODA i VOLKSWAGEN. Els clients els poden classificar segons l'àrea de treball:

- Àrea de negoci: El públic principal de l'aplicació és l'àrea de negoci encarregada de les activitats comercials relacionades amb la importació de productes. La nova aplicació NiX permetrà continuar treballant amb la generació de documentació antiga tot i utilitzar noves tecnologies. NiX transformarà automàticament aquests arxius sense que els usuaris hagin de fer cap canvi.
- Àrea de suport a negoci: Aquesta àrea ajuda a l'anterior amb les tasques quotidianes com configurar direccions o paràmetres. La nova aplicació els permetrà continuar treballant amb els sistemes que coneixen sense haver d'intervenir directament amb les noves tecnologies.
- Suport post servei (AMS): La nova aplicació millorarà el monitoratge diari dels processos i la resolució de problemes reduint la probabilitat d'aparició d'errors manuals, unificant els processos en un sol sistema i facilitant la identificació dels errors.

4. Especificació de requisits

En aquest apartat d'especificació s'estableixen els casos d'ús que volem afegir al sistema i els criteris tècnics per assegurar que l'aplicació compleix amb les necessitats i expectatives del client. Aquestes especificacions serviran per detallar tot el procés de desenvolupament i els resultats esperats un cop finalitzat cada procés.

4.1. Requisits funcionals

Començarem per llistar els nous casos d'ús a implementar en aquest projecte i la seva especificació. A la següent figura es mostra el diagrama de casos d'ús on es diferencia entre un usuari simple i un usuari amb permisos d'administrador el qual té algunes accions addicionals a l'usuari simple.

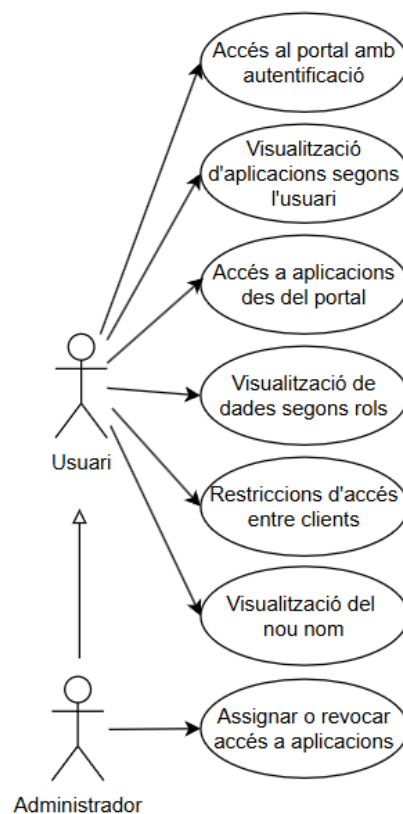


Figura 5: Diagrama de casos d'ús. Font: Elaboració pròpia.

A continuació, per a cada un dels casos d'ús indicats a la figura anterior s'indica l'actor implicat, l'objectiu, el disparador, les precondicions, el procediment i les postcondicions.

Cas d'ús	#01	Accés al portal amb autenticació
Actor	Usuari	
Objectiu	Accedir al portal mitjançant una autenticació de credencials personals.	
Disparador	L'usuari vol accedir al portal.	
Precondicions	L'usuari té credencials vàlides.	
Procediment	1. L'usuari accedeix a la pantalla d'inici de sessió del portal. 2. Introdueix usuari i contrasenya. 3. El sistema valida les credencials. 4. Si són vàlides, s'accedeix al portal.	
Postcondicions	L'usuari queda autenticat i pot accedir a les funcionalitats del portal.	

Taula 17: Cas d'ús Accés al portal amb autenticació. Font: Elaboració pròpia.

Cas d'ús	#02	Visualització d'aplicacions segons l'usuari
Actor	Usuari	
Objectiu	Visualitzar les aplicacions del portal a les quals l'usuari té accés.	
Disparador	L'usuari accedeix al portal.	
Precondicions	L'usuari està autenticat.	
Procediment	1. L'usuari inicia sessió. 2. El sistema consulta el llistat d'aplicacions a les quals l'usuari té accés. 3. Es mostra el llistat d'aplicacions corresponent.	
Postcondicions	L'usuari només veu les aplicacions a les quals té accés.	

Taula 18: Cas d'ús Visualització d'aplicacions segons l'usuari. Font: Elaboració pròpia.

Cas d'ús	#03	Accés a aplicacions des del portal
Actor	Usuari	
Objectiu	Accedir a les aplicacions a les quals té accés en el portal.	
Disparador	L'usuari selecciona una aplicació del llistat.	
Precondicions	L'usuari està autenticat i té accés a mínim una aplicació.	
Procediment	1. L'usuari accedeix al llistat d'aplicacions en el portal. 2. Si l'usuari només té accés a una aplicació, és redirigit a la pàgina principal d'aquesta.	

	3. Si l'usuari té accés a més d'una aplicació, selecciona l'aplicació a la qual vol accedir i és redirigit a la pàgina principal d'aquesta.
Postcondicions	L'usuari accedeix a una aplicació del portal.

Taula 19: Cas d'ús Accés a aplicacions des del portal. Font: Elaboració pròpia.

Cas d'ús	#04	Visualització de dades segons rols
Actor	Usuari	
Objectiu	Restringir la visualització i les accions (crear, modificar i eliminar) sobre dades segons els rols assignats a l'usuari.	
Disparador	L'usuari accedeix a l'aplicació de gestió de dades Commercial Cloud.	
Precondicions	L'usuari està autenticat, té accés a l'aplicació de gestió de dades Commercial Cloud amb certs rols assignats.	
Procediment	1. L'usuari accedeix a la pàgina principal de l'aplicació de gestió de dades Commercial Cloud. 2. El sistema verifica els rols i permisos de l'usuari. 3. Es permet o denega les accions i accessos a pantalles corresponents als rols.	
Postcondicions	L'usuari pot visualitzar i realitzar accions sobre les dades i pantalles a les quals té permís.	

Taula 20: Cas d'ús Visualització de dades segons rols. Font: Elaboració pròpia.

Cas d'ús	#05	Restriccions d'accés entre clients
Actor	Usuari	
Objectiu	Garantir que no es pot accedir a les dades d'altres clients.	
Disparador	L'usuari accedeix a dades.	
Precondicions	L'usuari està autenticat.	
Procediment	1. L'usuari navega per les pantalles tant del portal com de les aplicacions. 2. El sistema verifica el client al qual està associat l'usuari. 3. Només es mostren les dades del mateix client.	
Postcondicions	L'usuari només pot accedir a les dades del client al qual està associat.	

Taula 21: Cas d'ús Restriccions d'accés entre clients. Font: Elaboració pròpia.

Cas d'ús	#06	Visualització del nou nom
Actor	Usuari	

Objectiu	Convertir qualsevol menció de l'antic nom al nou nom.
Disparador	L'usuari accedeix al portal o a l'aplicació de gestió de dades Commercial Cloud.
Precondicions	L'usuari està autenticat.
Procediment	1. L'usuari navega per les pantalles tant del portal com de les aplicacions. 2. Es mostra el nou nom en qualsevol lloc on es faci menció.
Postcondicions	L'usuari només veu les mencions del nou nom tant en les pantalles com en les dades.

Taula 22: Cas d'ús Visualització del nou nom. Font: Elaboració pròpia.

Cas d'ús	#07	Assignar o revocar accés a aplicacions
Actor	Administrador	
Objectiu	Donar o treure accés a aplicacions als usuaris.	
Disparador	L'usuari administrador vol donar o treure accés a aplicacions a un usuari.	
Precondicions	L'usuari està autenticat i té accés a l'aplicació Commercial Cloud amb rols d'administrador.	
Procediment	1. L'usuari accedeix a l'aplicació Commercial Cloud. 2. El sistema verifica que té rols d'administrador. 3. Permet a l'usuari navegar per la pantalla de gestió d'aplicacions dels usuaris. 4. L'usuari accedeix a la pantalla de gestió d'aplicacions dels usuaris. 5. Selecciona l'usuari al qual vol afegir accés. 6. Selecciona o desselecciona del llistat d'aplicacions disponibles les aplicacions a les quals vol que l'usuari tingui accés. 7. El sistema actualitza les dades de l'usuari afectat.	
Postcondicions	El llistat d'aplicacions a les quals l'usuari afectat té accés es veurà modificat i l'usuari afectat veurà modificada la llista d'aplicacions en iniciar sessió al portal.	

Taula 23: Cas d'ús Assignar o revocar accés a aplicacions. Font: Elaboració pròpia.

4.1.1 Model conceptual del sistema

A continuació es mostren les classes i les associacions entre elles que constitueixen el model conceptual del sistema.

Primer, per una banda, per a poder gestionar les aplicacions a les quals un usuari té accés en el portal trobem l'associació entre *User* i *Application*. Més específicament, s'utilitza l'atribut

application_portal per a saber el llistat d'aplicacions del portal per a cada usuari. Les altres associacions estan relacionades amb l'aplicació de gestió de dades Commercial Cloud.

Dins de l'aplicació CC, els usuaris pertanyen a un *Cluster* i tenen un dels quatre tipus de *Cluster* associat: *Retailer* (concessionari), *Cluster* (agrupació), *Importer* (importador) o *Admin* (administrador). Aquest *Cluster* també té un tipus associat i conté unes marques concretes, les quals els usuaris del *Cluster* podran veure. A més, segons aquest *Cluster* i per a cada una de les seves marques, es poden assignar unes autoritzacions respecte a les accions sobre les dades. Aquestes autoritzacions (*AuthConfig*) són definides pels rols segons la marca i el tipus de *Cluster* i són respecte a cada microservei de l'aplicació, els quals representem amb la taula *Application*. Finalment, a cada rol se li assignen diversos permisos (*Authority*).

Restriccions d'integritat:

- Claus primàries: (*User*, id), (*Language*, id), (*ClusterType*, id), (*Application*, id), (*Authority*, id), (*Role*, id), (*Brand*, id), (*Cluster*, id).
- Cada *language_id* de cada taula de contingut pertany a un *Language* existent.
- Les dates han de ser vàlides i no poden ser posteriors al present.
- Per a totes les taules que aplica, no pot haver-hi elements d'un mateix tipus amb el mateix *code*.
- Les autoritzacions i *Cluster* assignat a un usuari han de ser del mateix tipus de *Cluster* que l'usuari.
- Les autoritzacions assignades a un usuari han de ser d'alguna de les marques que el *Cluster* que té associat conté.

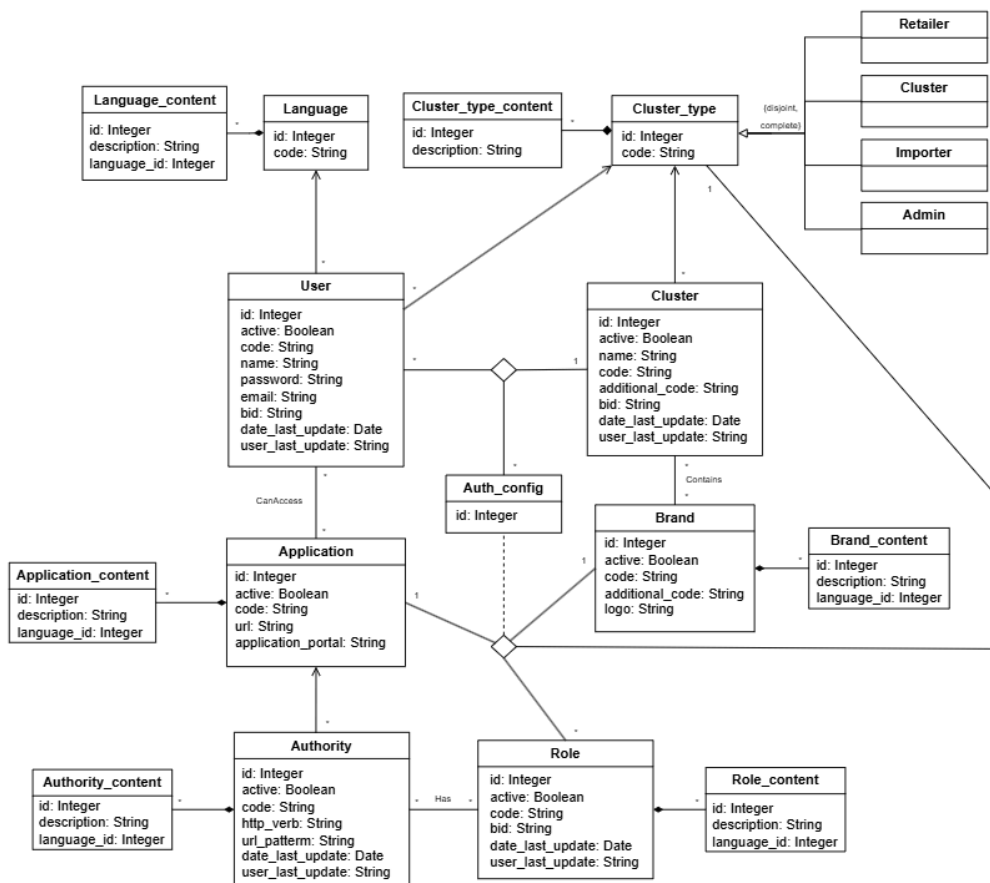


Figura 6: Model conceptual de dades. Font: Elaboració pròpia.

4.2. Requisits no funcionals

Per a l'especificació dels requisits no funcionals s'utilitzarà una adaptació de la plantilla Volere [18].

Identificació	#01
Tipus	10a. Aparència
Descripció	El nou contingut ha de mantenir una coherència visual amb el disseny de l'antiga versió de l'aplicació.
Justificació	Volem mantenir la coherència visual per facilitar la transició dels usuaris existents i garantir una experiència d'usuari familiar i contínua.
Criteris d'acceptació	El disseny, colors, tipografies i organització han de ser consistents amb l'antiga versió.

Taula 24: Requisit no funcional 1. Font: Elaboració pròpia.

Identificació	#02
Tipus	11a. Facilitat d'ús
Descripció	El sistema ha de ser fàcil d'utilitzar per a qualsevol usuari
Justificació	L'usuari ha de poder confiar en el fet que l'aplicació farà el que ell demana i el sistema l'ha de mantenir informat de procediment.
Criteris d'acceptació	El 80% dels usuaris no necessiten demanar a ajuda a tercer per a fer servir l'aplicació.

Taula 25: Requisit no funcional 2. Font: Elaboració pròpia.

Identificació	#03
Tipus	11b. Personalització i internacionalització
Descripció	L'aplicació ha d'estar disponible en diversos idiomes.
Justificació	El sistema ha de ser utilitzat per usuaris de diferents països.
Criteris d'acceptació	Traduccions completes i correctes en ambdues llengües.

Taula 26: Requisit no funcional 3. Font: Elaboració pròpia.

Identificació	#04
Tipus	11c. Aprenentatge
Descripció	L'usuari ha de poder efectuar les noves operacions sense formació especial.
Justificació	Garantir que l'usuari pot efectuar les noves operacions sense necessitat de formació especial és essencial per mantenir la productivitat i reduir costos de suport. També facilita l'adopció de noves funcionalitats i minimitza la resistència al canvi.
Criteris d'acceptació	El temps d'aprenentatge per les noves funcionalitats no hauria de superar els 5 min.

Taula 27: Requisit no funcional 4. Font: Elaboració pròpia.

Identificació	#05
Tipus	12a. Velocitat i latència
Descripció	Les operacions de consulta han de tenir un temps de càrrega inferior a 1 segon.
Justificació	El rendiment és clau per assegurar una experiència fluida i evitar la frustració dels usuaris, millorant la percepció general del sistema i la seva eficiència.

Criteris d'acceptació	Temps de resposta no ha superar 1 segon en el 95% de les consultes.
------------------------------	---

Taula 28: Requisit no funcional 5. Font: Elaboració pròpia.

Identificació	#06
Tipus	12a. Disponibilitat
Descripció	L'aplicació web ha de garantir un funcionament continu i estable, assegurant la seva disponibilitat en qualsevol moment.
Justificació	Una alta disponibilitat garanteix la confiança dels usuaris.
Criteris d'acceptació	La disponibilitat del sistema ha de ser de mínim 99,9%, fins i tot durant els moments d'ús intensiu.

Taula 29: Requisit no funcional 6. Font: Elaboració pròpia.

Identificació	#07
Tipus	12f. Capacitat
Descripció	L'aplicació web ha de suportar un gran nombre d'usuaris simultanis sense degradar el seu rendiment.
Justificació	S'ha d'assegurar la continuïtat operativa en entorns crítics o d'ús constant manté la productivitat i l'experiència d'usuari.
Criteris d'acceptació	La disponibilitat del sistema ha de ser de mínim 99,9%, fins i tot durant els moments d'ús intensiu.

Taula 30: Requisit no funcional 7. Font: Elaboració pròpia.

Identificació	#08
Tipus	15a. Accés
Descripció	El sistema ha de permetre la creació, consulta, modificació i eliminació de dades als usuaris autoritzats.
Justificació	S'ha de protegir la informació confidencial, prevenir l'accés no autoritzat i complir amb les normatives de seguretat i protecció de dades.
Criteris d'acceptació	El sistema ha de disposar d'un mecanisme segur d'autenticació i verificació de permisos per assegurar que cada usuari només pugui accedir a la informació i funcionalitats que li corresponen.

Taula 31: Requisit no funcional 8. Font: Elaboració pròpia.

Identificació	#09
Tipus	15b. Integritat
Descripció	L'aplicació ha de gestionar una separació estricta de dades per cada client.
Justificació	La seguretat és fonamental per protegir la informació confidencial i complir amb les normatives de protecció de dades, evitant riscos legals i reputacionals.
Criteris d'acceptació	Cap usuari pot accedir a dades d'altres clients.

Taula 32: Requisit no funcional 9. Font: Elaboració pròpia.

5. Disseny

5.1. Arquitectura del sistema

5.1.1. Arquitectura de microserveis

Com ja s'ha pogut anar intuïnt en apartats anteriors, l'aplicació té una arquitectura basada en microserveis. Consisteix a dividir el sistema en un conjunt de serveis petits, independents i autònoms on cada microservei s'encarrega d'una funcionalitat específica i es comunica amb la resta mitjançant diversos sistemes de missatgeria. Aquesta estructura contrasta amb el model monolític tradicional, on totes les funcionalitats estan agrupades en un únic bloc de codi.

Cada microservei es desplega de manera independent, la qual cosa permet actualitzacions, escalats i manteniments sense afectar la resta del sistema. També permet que cada servei pugui tenir el seu propi llenguatge de programació, base de dades i entorn. Aquesta flexibilitat facilita l'adaptació al canvi i l'escalabilitat de l'aplicació i millora la resiliència del sistema, ja que un error a un microservei no significa la caiguda completa de l'aplicació. No obstant això, també comporta reptes importants, com la gestió de la complexitat en la comunicació entre serveis, la necessitat d'eines de monitoratge avançades i l'aplicació rigorosa de polítiques de seguretat.

Els microserveis que formen part de l'aplicació que es desenvolupa en aquest projecte són els següents:

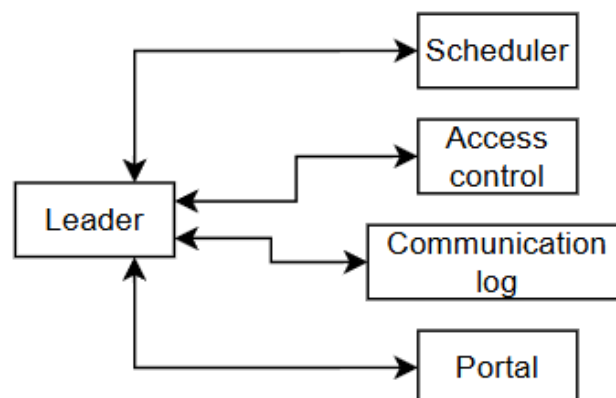


Figura 7: Microserveis. Font: Elaboració pròpia.

Leader

Aquest microservei té com a funció principal l'encaminament de totes les comunicacions cap al microservei corresponent dins de l'arquitectura del sistema. Actua com a punt central de gestió del trànsit, garantint que qualsevol interacció, tant amb serveis externs com entre microserveis interns, es realitzi de manera ordenada i segura. A més de la seva tasca d'encaminament, aquest component incorpora mecanismes de protecció per assegurar la integritat i la confidencialitat de les comunicacions. Concretament, bloqueja qualsevol intent de connexió no autoritzada i aplica determinats protocols de seguretat per prevenir accessos indeguts i possibles vulnerabilitats. Aquest microservei no té interfície, ja que només actua com a intermediari.

Portal

El microservei del portal té com a funció principal oferir una interfície centralitzada per a l'accés a les diferents aplicacions de VWGS. Aquest component actua com a punt d'entrada per als usuaris, proporcionant una experiència unificada. A més de la seva funció com a portal, aquest microservei incorpora els binaris necessaris per al *frontEnd* tant del mateix portal com de la resta d'aplicacions del portal. Això garanteix que la capa de presentació estigui disponible i actualitzada, facilitant el desplegament i reduint la dependència d'elements externs. D'aquesta manera, el portal no només centralitza l'accés, sinó que també assegura la coherència visual i funcional de tot l'ecosistema.

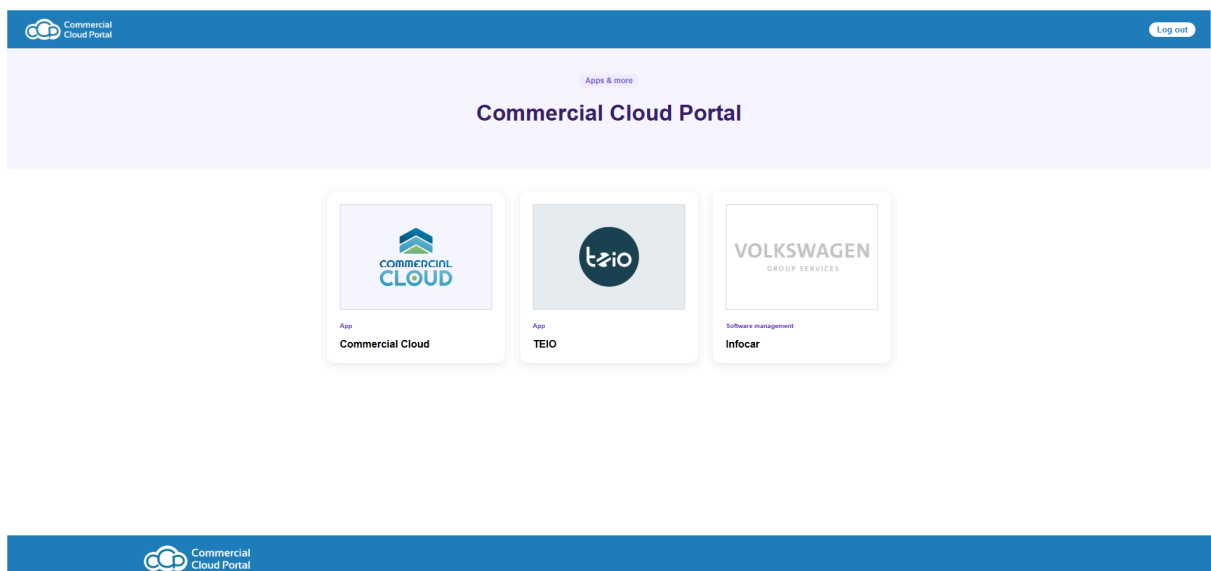


Figura 8: Pantalla del portal. Font: Elaboració pròpia.

Access Control

El microservei *Access Control* té com a objectiu principal la gestió d'autenticació d'usuaris i els seus permisos dins de l'aplicació. Aquest component s'encarrega de verificar la identitat dels usuaris mitjançant mecanismes d'autenticació, determina els permisos associats a cada usuari o servei segons rols i polítiques definides així com les agrupacions a les quals pertany i gestiona la creació i manteniment d'aquests rols. Aquest microservei és essencial per mantenir la confidencialitat i protecció de dades.

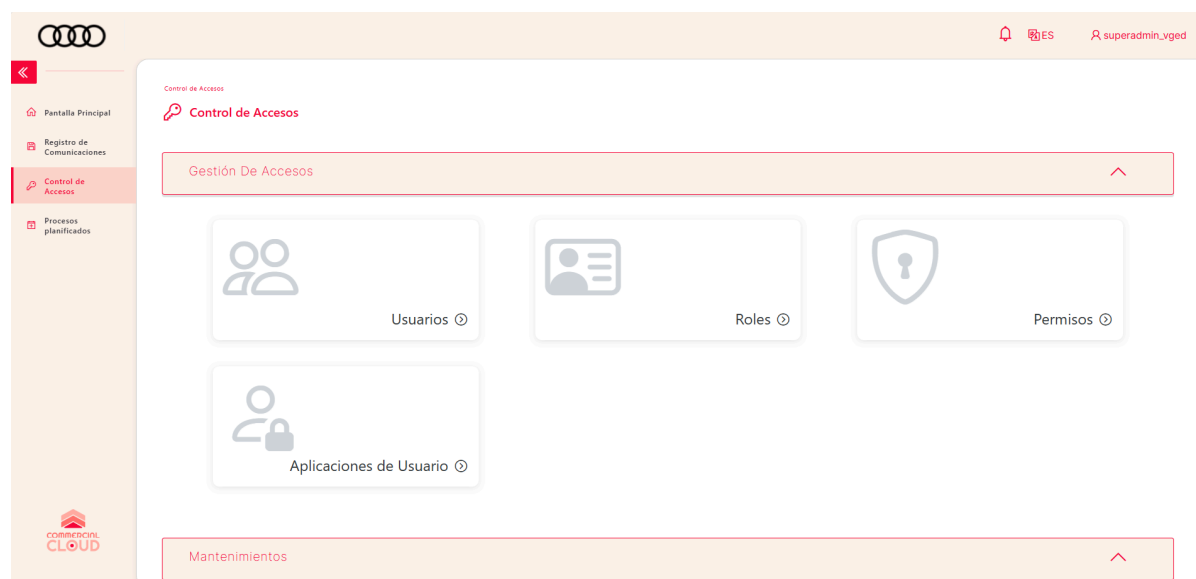


Figura 9: Pantalla d'Access Control. Font: Elaboració pròpia.

Communication Log

Tot i no haver-hi modificacions en aquest microservei durant el projecte, també es separarà segons l'importador i s'haurà de posar en marxa, ja que forma part de l'aplicació. El microservei *Communication Log* té com a finalitat principal registrar totes les comunicacions que es produeixen dins del sistema, tant entre microserveis com amb serveis externs. Aquest component actua com un mecanisme de traçabilitat i auditoria, assegurant que cada interacció quedi documentada per a anàlisi posterior, resolució d'incidències i compliment normatiu.

Registro de Comunicaciones / Listado de Comunicaciones

Listado de Comunicaciones

Código: Selecciona un registro | Origen: Selecciona un registro | Destino: Selecciona un registro

Cuerpo: Cuerpo | Resultado: Selecciona un registro | ☐ Ocultar vacíos

Rango de Fechas: [] | Aplicar

*Pantalla donde el usuario puede revisar las comunicaciones en tiempo real que pasan por CC ya sean recibidas o enviadas y el contenido de los mensajes/archivos.

ACCION...	CÓDIGO	ORIGEN	DESTINO	FECHA DE INICIO	FECHA DE FINALIZACIÓN
...	Invoice Order Generation	ONE SALES	SLI	01-07-2025 12:25:15	01-07-2025 12:25:21
...	Down Payment Cancellation	ONE SALES	GIS	01-07-2025 12:25:13	01-07-2025 12:25:21
...	Offer Modification	ONE SALES	GIS	01-07-2025 12:25:08	01-07-2025 12:25:14
...	Offer Modification	ONE SALES	GIS	01-07-2025 12:25:08	01-07-2025 12:25:13
...	Down Payment Entry	ONE SALES	GIS	01-07-2025 12:25:08	01-07-2025 12:25:19
...	Down Payment Entry	ONE SALES	GIS	01-07-2025 12:25:08	01-07-2025 12:25:19
...	Offer Modification	ONE SALES	GIS	01-07-2025 11:10:16	01-07-2025 11:10:16

Figura 10: Exemple de llistat de comunicacions. Font: Elaboració pròpia.

A més, ofereix la funcionalitat de generació de gràfiques i panells interactius que representen estadístiques de comunicació, volum de trànsit, temps de resposta i incidències, facilitant una anàlisi més intuïtiva i ràpida.

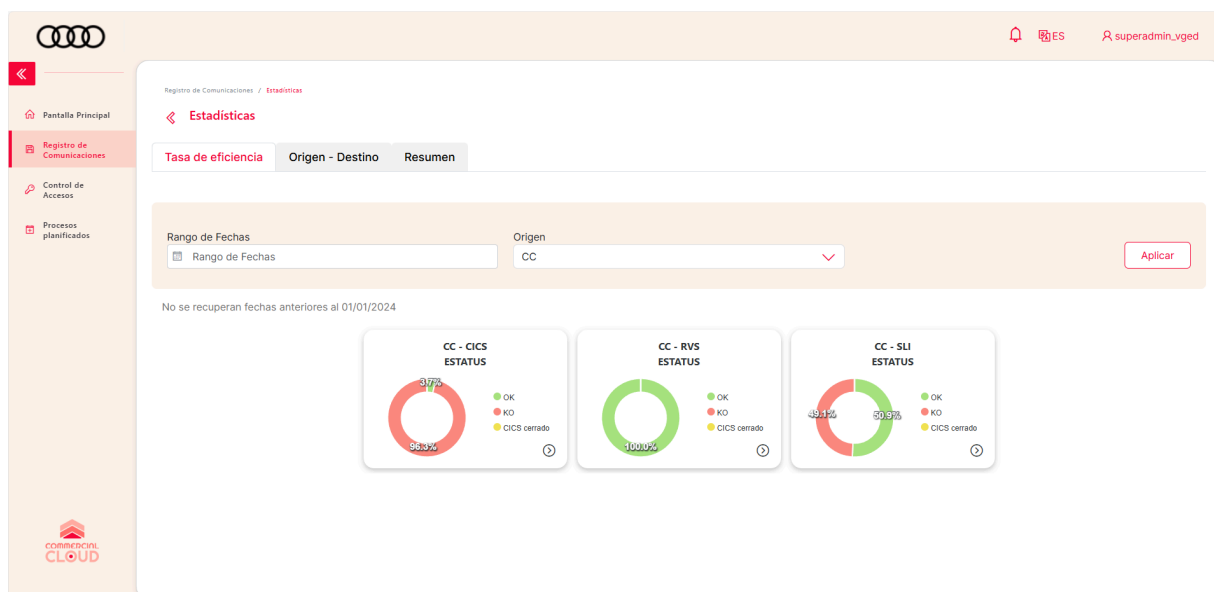


Figura 11: Exemple d'estadístiques de comunicacions. Font: Elaboració pròpia.

Scheduler

Per acabar, aquest microservei tampoc ha patit cap modificació. Té com a funció principal la planificació i programació de processos dins del sistema. Aquest component permet definir, automatitzar i executar tasques en moments específics, garantint que les operacions s'executin de manera ordenada i segons els requisits temporals definits. Alguns exemples d'activitats que es poden programar són tasques de manteniment com la neteja de registres

antics o tasques de monitoratge com proves periòdiques per verificar l'estat dels microserveis.

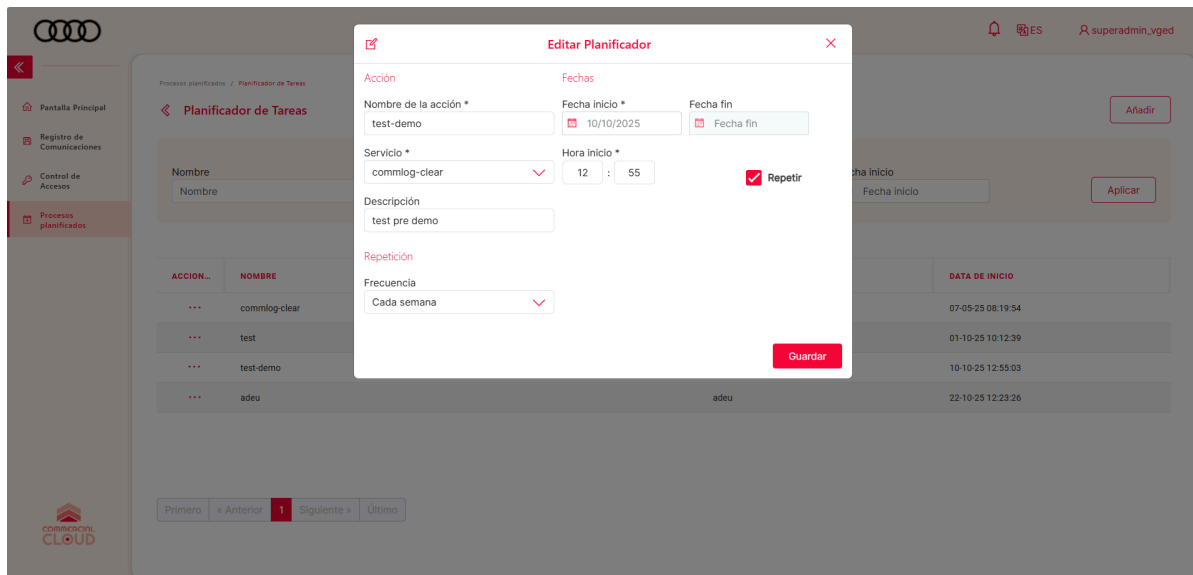


Figura 12: Exemple d'una tasca. Font: Elaboració pròpia.

Adicionalment, existeixen els microserveis de *Ordering* i *GSB*. Aquests dos microserveis s'encarreguen de les crides cap a serveis externs relacionades amb la venda d'automòbils. Tot i que aquests microserveis estan involucrats en les tasques de reorganització de microserveis, específicament **[RM1] Modificar consultes a GSB des de Ordering**, no formen part del cos principal de l'aplicació i es gestionen per separat a la resta de microserveis. A més, el nostre equip no és l'encarregat del desenvolupament ni del manteniment d'aquests microserveis. Tampoc tenen interfície d'usuari en l'aplicació que es desenvolupa en aquest projecte, per tant, no els tindrè en compte com a microserveis de l'aplicació ni entraré en detall sobre el seu funcionament.

5.1.2. Arquitectura Cloud

Com s'ha mencionat anteriorment, l'aplicació també aplica una arquitectura de núvol o *cloud*. És a dir, l'aplicació està dissenyada, desplegada i executada utilitzant serveis al núvol (*cloud computing*) en lloc de servidors locals o físics propis. Els recursos necessaris per a l'aplicació com la xarxa, la base de dades o els servidors s'ofereixen per Internet. L'ús d'aquest tipus d'arquitectura permet alta escalabilitat, disponibilitat i accessibilitat, ja que la gestió dels recursos es pot fer segons les necessitats i en qualsevol lloc amb connexió a Internet. També redueix la mantenibilitat, gestionada pels proveïdors dels recursos, i sense grans despeses, pel fet que es paga pels recursos consumits.

El proveïdor per aquest projecte és Amazon Web Services el qual té un ampli catàleg de serveis i recursos *cloud*.

Els usuaris accedeixen a l'aplicació des de navegadors web. Segons el domini al qual s'accedeixi, l'usuari pot accedir al Portal de SEAT o VGED que actuen com a punts d'entrada al sistema. En cada portal es realitzen els controls d'autenticació i autorització pertinents amb ajuda del *Leader* el qual decideix a quin domini funcional s'ha d'encaminar cada usuari. Una vegada superada la capa del Portal, s'accedeix als microserveis interns de l'aplicació. Finalment, els microserveis estan connectats a una base de dades compartida desacoblada per microservei i client. Hi ha un esquema per a cada microservei i cada client té el seu grup d'esquemes per microservei.

Tenint en compte les dues arquitectures explicades, a la següent figura es mostra un resum de com és l'arquitectura del sistema que s'implementarà.

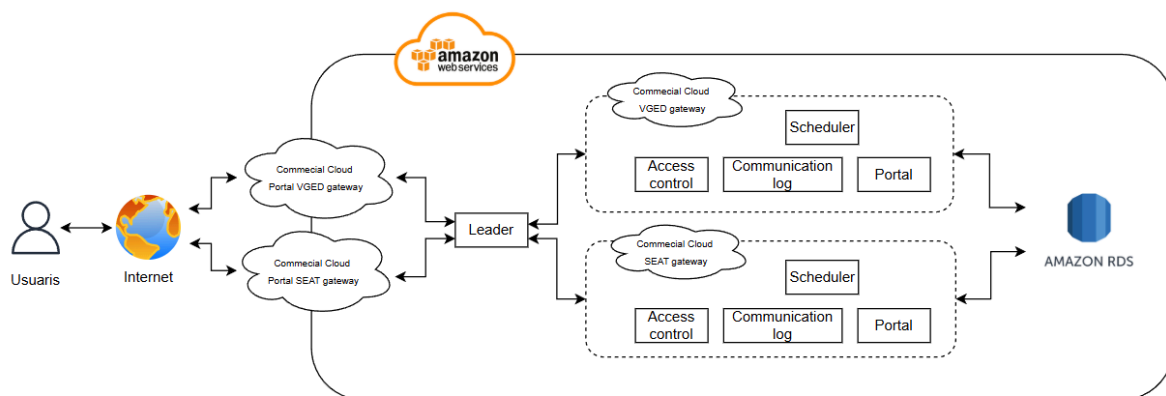


Figura 13: Arquitectura del sistema. Font: Elaboració pròpia.

5.2. Patrons de disseny

L'aplicació segueix el patró Client-Servidor el qual organitza el codi en dos components diferenciats: el client, que formula sol·licituds de servei, i el servidor, que les rep, les processa i retorna respostes. Aquesta separació de responsabilitats permet que el client es focalitzi en la presentació i en l'experiència d'usuari, mentre que el servidor se centra en la lògica de negoci, la seguretat, la integritat de les dades i la persistència. La comunicació entre els dos components es fa mitjançant un intercanvi de missatges seguint protocols de xarxa estàndards i fent ús de formats de representació de dades interoperables.

En el present projecte, el client s'ha desenvolupat amb Angular, implementant una *Single-Page Application* (SPA) [19] que renderitza vistes dinàmiques i gestiona la navegació interna sense haver de recarregar la pàgina, millorant la fluïdesa d'ús. Pel servidor s'ha

utilitzat Spring Boot, que proporciona una plataforma per exposar una API REST [20]. El protocol d'intercanvi de missatges que es fa servir en aquest cas és HTTP [21] i el format de dades emprat és JSON [22].

A continuació, s'explica més detalladament l'estructura de cada component, els seus funcionaments i el procés de comunicació que s'estableix entre ells.

5.2.1. Client

El client, el que també anomenem *frontEnd*, s'encarrega de la presentació i l'experiència d'usuari de l'aplicació encapsulant la lògica de la interfície i la coordinació de les crides a l'API del servidor. Angular aplica el patró MVVM (*Model-View-ViewModel*), ja que la seva arquitectura està basada en components i serveis. El patró MVVM és un derivat del clàssic patró Model-Vista-Controlador, pensat per a la implementació d'interfícies d'usuari.

A continuació, s'explica com es distribueixen els components del patró MVVM dins del codi d'aquest projecte

El component que anomenem *View*, Vista en català, està format pels fitxers HTML i SCSS de cada component que conformen l'aplicació. Són els responsables de la presentació de les pantalles i de capturar les interaccions de l'usuari com els clics o les entrades de text. El fitxer HTML defineix l'estructura de la pantalla i fitxer SCSS s'ocupa de l'estilització i la presentació visual.

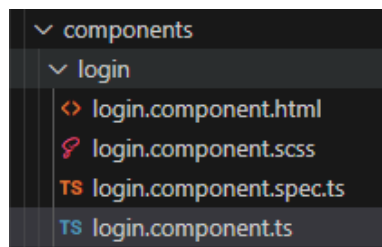


Figura 14: Fitxers pel component login. Font: Elaboració pròpia.

```

<form [formGroup]="formGroup" (ngSubmit)="login()">
  <div class="alert alert-danger" *ngIf="hasError">
    <div class="container">
      <div class="row align-items-center">
        <div class="col-lg-2 col-xs-6">
          <div>
            <i class="la la-3x la-exclamation-triangle"></i>
          </div>
        </div>
        <div class="col-md">
          <h6 class="alert-heading">{{'Error'}}</h6>
          <p class=""><small>{{'Your password is incorrect or this account doesn\'t exist.'}}</small></p>
        </div>
      </div>
    </div>
  </div>
  <label for="username">User</label>
  <input type="text" id="username" [placeholder]="\'Enter your username\'" formControlName="username" required />

  <label for="password">Password</label>
  <input type="password" id="password" [placeholder]="\'.....\'" formControlName="password" required />
  <!-- <a href="#" class="forgot-link">Forgot password?</a> !-->

  <button type="submit" class="login-button">Log in</button>
</form>

```

Figura 15: HTML pel formulari de login. Font: Elaboració pròpia.

```

form {
  display: flex;
  flex-direction: column;
  text-align: left;

  label {
    font-size: 0.9rem;
    margin: 0.5rem 0 0.2rem;
  }

  input {
    padding: 0.5rem;
    border: 1px solid #ccc;
    border-radius: 6px;
    margin-bottom: 0.5rem;
  }

  .forgot-link {
    font-size: 0.8rem;
    color: #6a1b9a;
    text-align: right;
    display: block;
    margin-bottom: 1rem;
    text-decoration: none;
  }

  .login-button {
    background-color: #1166a4;
    color: white;
    padding: 0.6rem;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
    font-size: 1rem;
    margin-top: 2rem;
  }
}

```

Figura 16: SCSS pel formulari de login. Font: Elaboració pròpia.

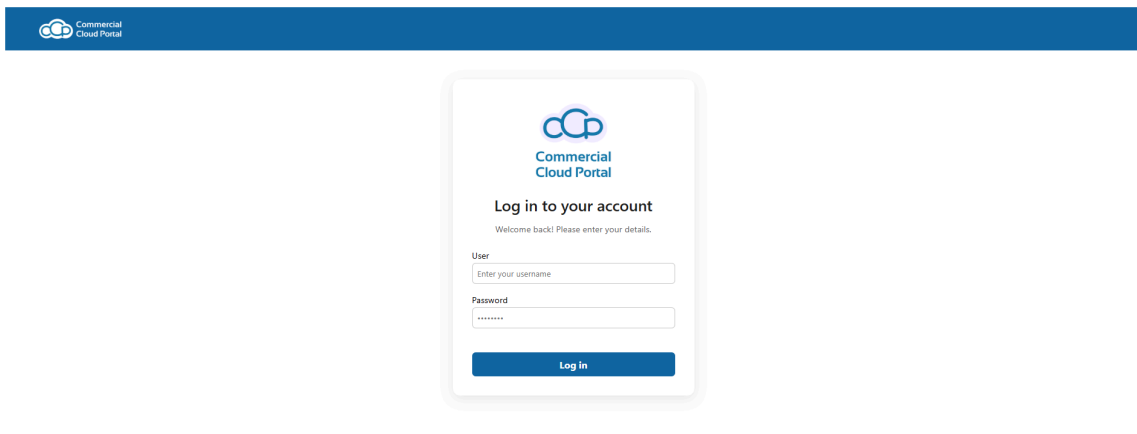


Figura 17: Resultat de la interfície del login. Font: Elaboració pròpia.

El component *ViewModel* agrupa els fitxers TS dels components i els serveis que tenen associats. Cada fitxer TS defineix el comportament del component de l'aplicació al que correspon encarregant-se del processament de dades i les validacions. Per l'altra banda, els serveis són els responsables d'orquestrar les crides HTTP cap al servidor. El *ViewModel* actua com a intermediari entre la *View* i el *Model*, proporcionant les dades que la vista necessita i gestionant les accions de l'usuari.

```

ngOnInit(): void {
  // Check if user is already logged
  this.authService.getUserConnectedPortal().subscribe((u) => {
    // If user is connected, navigate to portal
    if (u) {
      console.log("logged");
      this.router.navigate(['/portal']);
    }
  });
  this.generateFormGroup();
}

generateFormGroup(): void {
  this.formGroup = new FormGroup({
    username: new FormControl(undefined, Validators.required),
    password: new FormControl(undefined, Validators.required),
  });
}

login(): void {
  if (this.formGroup.valid) {
    let idx = environment.serverUrl.lastIndexOf('/');
    let env = environment.serverUrl.substring(idx + 1);
    env += '_._.';
    this.hasError = false;
    // Call login method from AuthService
    this.authService
      .login(
        env + this.formGroup.controls['username'].value,
        this.formGroup.controls['password'].value
      )
      .subscribe(
        () => {
          this.router.navigateByUrl('/portal');
        },
        () => {
          this.hasError = true;
        }
      );
  }
}

```

Figura 18: Operacions pel formulari de login. Font: Elaboració pròpia.

```

login(username: string, password: string): Observable<UserAuth> {
  const params = new HttpParams({
    fromObject: {
      username,
      password,
    },
  });

  const httpOptions = {
    headers: new HttpHeaders({
      "Content-Type": "application/x-www-form-urlencoded",
    }),
  };

  return this.http
    .post<UpdateCtx>(`${environment.gateway}/auth/login`, params, httpOptions)
    .pipe(
      map((uc: UpdateCtx) => {
        this.userctx = uc;
        this.setTokenAndUser(uc.nixTokenValue, uc);
        return uc.user;
      })
    );
}

```

Figura 19: Operació HTTP del login. Font: Elaboració pròpia.

Per acabar, els *Models* són les classes TypeScript que defineixen el contracte de dades consumit per la vista i pel *ViewModel*. Aquestes classes permeten a la *View* saber quines dades s'han de mostrar i al *ViewModel* saber com manipular-les, tot això seguint una coherència gràcies al tipatge fort que aplica Angular el qual comprova que les dades compleixen amb el format establert.

```

import { Group } from './group.model';
import { Application } from './application.model';
import { Language } from './language.model';

export class UserAuth {
  id!: number;
  code!: string;
  active!: boolean;
  group!: Group;
  applications!: Application[];
  language!: Language;
  currentConfig: any = {};
}

```

Figura 20: Model de userAuth. Font: Elaboració pròpia.

La manera amb la qual Angular connecta la *View* amb el *ViewModel* és implementant el *data binding*. Específicament, per connectar els fitxers HTML amb els seus corresponents fitxers TS. Es creen cicles de detecció de canvis (*change detection*) per sincronitzar la vista amb el component. Alguns *data binding* fan que quan el component actualitzi una variable,

la vista es renderitzi de nou per a mostrar el canvi mentre que altres estan pensats per què quan l'usuari modifiqui una entrada, s'actualitzi el model intern.

Per a mostrar millor com funciona el *data binding*, es mostren exemples d'aplicació de cada un dels quatre tipus de *data binding* en Angular que es poden trobar en el codi d'aquesta aplicació.

- Interpolació (*Interpolation* `{{}}`): Permet inserir valors de variables o resultats d'expressions JavaScript senzilles dins de l'HTML. Se substitueix el contingut pel valor durant l'execució. Només permet mostrar informació del component a la vista i aquesta informació es limita a resultats de propietats, expressions matemàtiques, crides a mètodes simples, etc. No pot contenir ordres complexos com assignacions o bucles. Un exemple molt bàsic seria:

```
<label class="align-self-center">{{ errorMessage }}</label>
```

Angular substituirà `{{ errorMessage }}` pel contingut que tingui aquesta variable en el component.

- Enllaç de propietats (*Property Binding* `[]`): Associa una propietat de l'element HTML a una expressió del component. Si el valor de la propietat canvia en el component, la vista s'actualitza. És a dir, qualsevol canvi en el model que es troba en component actualitza la vista automàticament. Un exemple que es pot troba en molts HTML del codi és:

```
<app-data-table [columnDefs]="columnDefs" [rowData]="rowData">
</app-data-table>
```

En aquesta línia per mostrar una taula, quan les variables `columnDefs` i `rowData` canvien, a la pantalla s'actualitza el contingut i la definició de les columnes de la taula.

- Enllaç d'esdeveniments (*Event Binding* `()`): Relaciona esdeveniments de la vista com clics del ratolí i del teclat o moviments del ratolí amb mètodes del component. Quan es produeix l'esdeveniment, Angular executa la funció especificada al *ViewModel*. L'exemple que es mostra a continuació és la codificació d'un botó per guardar informació.

```
<button [disabled]="formUser.invalid || formCluster.invalid
|| formRole.invalid" class="btn btn-primary"
(click)="save()">
{{ "save" | translate }}
```

```
</button>
```

En les propietats afegides al botó, el *data binding* aplicat determina que s'executi la funció `save()` definida en el component quan es premi aquest botó.

- Enllaç bidireccional (*Two-Way Binding* `[(())]`): Sincronitza el model amb la vista de manera que els canvis a la vista automàticament modifiquen el model i, qualsevol canvi al model es reflecteix a la vista. És una combinació entre els enllaços de propietats i d'events.

```
<input class="form-control border-0" placeholder="Type to  
search..." [(ngModel)]="availableInputValue"  
(input)="onChangeAvailableInput()" >
```

Per aquest camp d'un formulari, si l'usuari escriu en aquest camp, el valor de `availableInputValue` s'actualitza automàticament i, si el valor de `availableInputValue` canvia en el component, el text del camp també canvia.

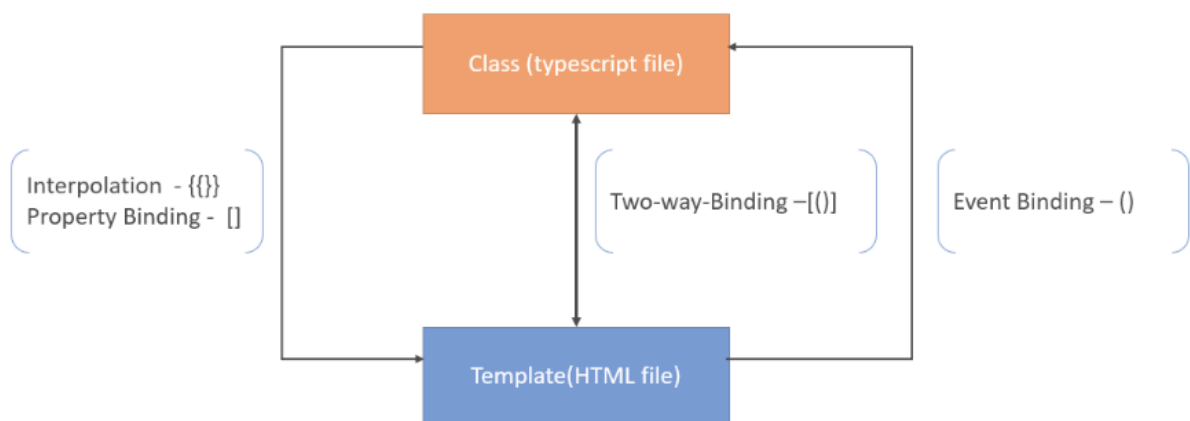


Figura 21: Flux de comunicacions dels data bindings. Font: <https://miro.medium.com>.

La construcció i execució de tot el *frontEnd* es duu a terme emprant Angular CLI el qual genera una versió optimitzada i minimitzada de l'aplicació. El resultat d'aquest procés és un conjunt de codi binari necessari pel desplegament. Una vegada obtinguts els binaris, aquests s'afegeixen com a contingut estàtic en el microservei del portal des d'on es desplegaran i executaran.

5.2.2. Servidor

Pel servidor, el *backEnd*, Spring Boot organitza el codi seguint el patró a capes amb les quals se separen responsabilitats: la capa de presentació rep les peticions i construeix les respostes, la capa de lògica de negoci aplica les regles del domini, la capa d'accés a dades interactua

amb la base de dades i la capa de domini defineix el model de dades. S'hi afegeix una capa transversal de contracte que separa el model intern de la representació externa de l'API. Aquesta estructura clarifica rols, evita acoblaments innecessaris i facilita provar, reutilitzar i mantenir el codi amb el pas del temps.

Capa de presentació (*Controllers*)

L'accés a les dades i la gestió de la lògica en aquest projecte es duu a terme mitjançant operacions REST, que estructuren i normalitzen la comunicació amb la capa de dades. Els controladors, per mitjà de funcions específiques, processen les peticions rebudes, gestionen la validació de la informació i canalitzen les sol·licituds cap als mòduls o serveis corresponents de la capa de dades, assegurant-se que les operacions requerides s'executin amb la màxima eficàcia. Un cop s'han obtingut els resultats d'aquestes operacions a la capa de dades, la capa d'aplicació s'encarrega de retornar-los de forma estructurada i segura cap a la capa de presentació. L'existència d'aquesta capa garanteix una separació clara de responsabilitats entre les capes de dades i presentació, facilitant la gestió eficient de les operacions de l'aplicació.

Dins de cada microservei hi ha una classe *Controller* per a cada model.

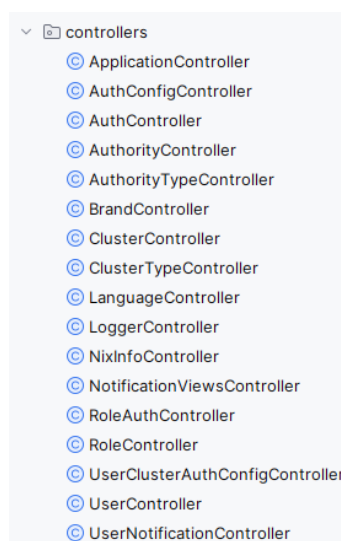


Figura 22: Controladors del microservei Access Control. Font: Elaboració pròpia.

En cada un d'aquests *Controllers* es defineixen les operacions REST necessàries per cada cas.

```

@GetMapping("/filter") no usages  ⚡ Daniel Españañero +2
public ResponseModel findLanguageByParams(@PathParam("description") String description,
                                           @PathParam("code") String code) throws DefaultAccessControlException {
    logger.debug("findLanguageByParams() params => description: {}, code {}", description, code);
    return new ResponseModel( code: null, languageService.findLanguagesByParams(code, description));
}

@PostMapping("/create") no usages  ⚡ Daniel Españañero +1 *
public ResponseModel createLanguage(@Validated(Create.class) @RequestBody LanguageDTO language)
    throws AccessControlFKViolationException, DefaultAccessControlException {
    logger.debug("createLanguage() params => language: {}", language);
    return new ResponseModel( code: "ACCTRL-0080", languageService.saveLanguage(language));
}

@PutMapping("/update") no usages  ⚡ Daniel Españañero +1 *
public ResponseModel updateLanguage(@Validated(Update.class) @RequestBody LanguageDTO language)
    throws AccessControlFKViolationException, DefaultAccessControlException {
    logger.debug("updateLanguage() params => language: {}", language);
    return new ResponseModel( code: "ACCTRL-0081", languageService.saveLanguage(language));
}

@DeleteMapping("/delete/{id}") no usages  ⚡ Daniel Españañero +2 *
public ResponseModel deleteLanguage(@PathVariable("id") Long id)
    throws AccessControlFKViolationException, AccessControlDataNotFoundException, DefaultAccessControlException {
    this.languageService.deleteLanguage(id);
    logger.debug("deleteLanguage() params => id: {}", id);
    return new ResponseModel( code: "ACCTRL-0084", data: null);
}

```

Figura 23: Exemple d'operacions REST pel model Language. Font: Elaboració pròpia.

Capa de lògica de negoci (Services)

Els serveis ofereixen la interfície i la implementació de tota la lògica de negoci de l'aplicació. Una vegada es fa la petició REST en els controladors, aquestes criden a les funcions corresponents per a obtenir el resultat de la petició. A través d'aquestes funcions s'executen operacions complexes, combinant la validació, el processament i el flux de dades entre els diferents components.

```

public interface ILanguageService { 8 usages 1 implementation  🤖 Daniel Españadero +2
    Long getDefaultLanguageId(); 3 usages 1 implementation  🤖 Bruno Calvo

    LanguageBasicDTO getBasicLanguageById(Long languageId) throws AccessControlDataNotFoundException; 4 usages 1 implementation  🤖 Daniel Españadero

    List<LanguageDTO> getLanguageList() throws DefaultAccessControlException; 3 usages 1 implementation  🤖 Daniel Españadero

    List<LanguageDTO> findLanguagesByParams(String code, String description) throws DefaultAccessControlException; 2 usages 1 implementation  🤖 Daniel Españadero

    LanguageDTO saveLanguage(LanguageDTO newLang) throws AccessControlFKViolationException, DefaultAccessControlException; 6 usages 1 implementation  🤖 Daniel Españadero

    void deleteLanguage(Long id) throws AccessControlFKViolationException, DefaultAccessControlException, AccessControlDataNotFoundException;

    LanguageBasicDTO getDefaultLanguage(); 1 usage 1 implementation  🤖 Bruno Calvo

    LanguageBasicDTO getBasicLanguageByCode(String code) throws AccessControlDataNotFoundException; 2 usages 1 implementation  🤖 Daniel Españadero

    LanguageBasicDTO getBasicLanguageByIsoCode(String code); 2 usages 1 implementation  🤖 Daniel Españadero

    List<LanguageBasicDTO> getBasicLanguageByCodeList(List<String> codes); 2 usages 1 implementation  🤖 Daniel Españadero

    List<LanguageBasicDTO> getAllLanguage() throws DefaultAccessControlException; 2 usages 1 implementation  🤖 Daniel Españadero
}

```

Figura 24: Interfície del servei de Language. Font: Elaboració pròpia.

```

@Override 6 usages  🤖 rtapastau +3
public LanguageDTO saveLanguage(LanguageDTO newLang) throws AccessControlFKViolationException, DefaultAccessControlException {
    logger.debug("saveLanguage() params => newLang: {}", newLang);

    try {
        Language langToSave = languageMapper.toEntity(newLang);
        LanguageDTO langSaved = languageMapper.toDTO(languageRepository.save(langToSave));
        logger.debug("saveLanguage() param return => langSaved: {}", langSaved);
        return langSaved;
    } catch (DataIntegrityViolationException e) {
        String errorCode = "ACCCTRL-0083";
        Throwable rootCause = e.getRootCause();
        if (rootCause instanceof SQLIntegrityConstraintViolationException sqlIntegrityConstraintViolationException) {
            int sqlErrorCode = sqlIntegrityConstraintViolationException.getErrorCode();
            if (sqlErrorCode == SQLErrorConstants.DUPLICATE_ENTRY_ERROR) {
                errorCode = "ACCCTRL-0079";
            }
        }
        logger.error(e.getMessage());
        throw new AccessControlFKViolationException(errorCode);
    } catch (Exception e) {
        logger.error(e.getMessage());
        throw new DefaultAccessControlException();
    }
}

```

Figura 25: Exemple d'operació en el servei de Language. Font: Elaboració pròpia.

Capa de domini (Models)

Els models representen l'estructura interna i persistent de les entitats del sistema reflectint idènticament la informació emmagatzemada. Els models han de tenir una correspondència directa amb l'esquema de la base de dades, ja que serveixen de base per la creació, consulta i actualització de la informació emmagatzemada. S'hi defineixen els mètodes *setters* i *getters* que assegurin l'accés i la modificació segura i controlada dels atributs de les entitats.

```

@Entity
@Table(name = "language")
public class Language extends I18nEntity<LanguageContent> {

    @Id 3 usages
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(name = "code", unique = true) 3 usages
    private String code;

    @Column(name = "iso_code", unique = true) 2 usages
    private String isoCode;

    @Column(name = "mbv") 2 usages
    private String mbv;

    @Column(name = "region") 2 usages
    private String region;

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getCode() { return code; }

    public void setCode(String code) { this.code = code; }

    public String getIsoCode() { return isoCode; }

    public void setIsoCode(String isoCode) { this.isoCode = isoCode; }

    public String getMbv() { return mbv; }

    public void setMbv(String mbv) { this.mbv = mbv; }

    public String getRegion() { return region; }

    public void setRegion(String region) { this.region = region; }
}

```

Figura 26: Model de Language. Font: Elaboració pròpia.

A aquest model li correspon un altre anomenat *LanguageContent* el qual conté la descripció de cada idioma en tots els idiomes disponibles. Això s'aconsegueix amb la classe genèrica *I18nContent* que encapsula tots els textos, descripcions, etiquetes i altres recursos textuais o multimèdia que han de ser adaptats a diversos idiomes. Ens serveix com a base per gestionar el contingut internacionalitzat.

```

@Entity 14 usages  ⚙️ rta pastau
@Table(name = "language_content")
@Audited
public class LanguageContent extends I18nContent<Language> {

    @Id 2 usages
    @GeneratedValue
    private Long id;

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

}

```

Figura 27: Model de LanguageContent. Font: Elaboració pròpia.

Capa de contracte (DTO/Mappers)

Els *Data Transfer Objects* (DTOs) actuen com a contenidors d'informació que permeten estructurar les dades necessàries per a cada petició, proporcionant un format personalitzat en funció dels requisits de resposta. Aquestes classes també estan dotades de *setters* i *getters*.

A la següent figura es veurà un exemple d'un DTO per a la petició d'un idioma segons el seu identificador. La principal diferència que es podrà observar és l'aparició de l'atribut *description*. Aquest atribut s'obté de la taula *LanguageContent* com veurem més endavant.

```
public class LanguageDTO extends I18nEntityDTO<LanguageContentDTO> {  @ rtapastau

    Long id; 3 usages
    String code; 3 usages
    String isoCode; 2 usages
    String description; 2 usages
    String mbv; 2 usages
    String region; 2 usages

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getCode() { return code; }

    public void setCode(String code) { this.code = code; }

    public String getIsoCode() { return isoCode; }

    public void setIsoCode(String isoCode) { this.isoCode = isoCode; }

    public String getDescription() { return description; }

    public void setDescription(String description) { this.description = description; }

    public String getMbv() { return mbv; }

    public void setMbv(String mbv) { this.mbv = mbv; }

    public String getRegion() { return region; }

    public void setRegion(String region) { this.region = region; }

    @Override  @ rtapastau
    public String toString() { return "LanguageDTO [id=" + id + ", code=" + code + "]; }
}
```

Figura 28: Exemple d'un DTO per Language. Font: Elaboració pròpia.

Els *Mappers* són els responsables de la conversió entre els DTOs i els models. La seva funció és garantir la transformació correcta i eficient de les dades, segons si es tracta de transferir informació cap a la capa de presentació o bé cap a la capa de persistència.

En l'exemple es veu la conversió del model *Language* al *DTO* mostrat en la figura anterior. L'atribut *descriptions* del *DTO* el completa amb la descripció que obté del model *LanguageContent*.

```
public LanguageDTO toDTO(Language language) { 8 rtapastau +1
    Long lang = userDetails.getLanguage().getId();
    Long defaultLanguage = 1L;

    LanguageDTO dto = new LanguageDTO();
    dto.setId(language.getId());
    dto.setCode(language.getCode());
    dto.setIsoCode(language.getIsoCode());
    dto.setMbv(language.getMbv());
    dto.setRegion(language.getRegion());

    if (language.getContents() != null) {
        Map<Long, LanguageContentDTO> mapdto = language.getContents().entrySet()
            .stream().collect(Collectors.toMap( Entry<Long, LanguageContent> e -> e.getValue().getLanguage(),
                Entry<Long, LanguageContent> e -> contentMapper.toDTO(e.getValue())));
        dto.setContents(mapdto);

        if (language.getContents().containsKey(defaultLanguage)) {
            dto.setDescription(language.getContents().get(defaultLanguage).getDescription());
        }
        if (language.getContents().containsKey(lang)) {
            dto.setDescription(language.getContents().get(lang).getDescription());
        }
    }

    return dto;
}
```

Figura 29: Operació de conversió de Model a DTO. Font: Elaboració pròpia.

Capa d'accés a dades (*Repositories*)

Finalment, els *Repositories* són encarregats d'efectuar consultes sobre la base de dades. Proporcionen operacions per defecte, com ara *findByCode*, de manera que es facilita el control de l'accés als registres i, quan les necessitats de consulta són més avançades, aquestes operacions se supplementen amb la definició explícita de les *SQL queries* corresponents.

```
public interface LanguageRepository extends JpaRepository<Language, Long> { 8 usages 8 rtapastau +1

    Optional<Language> findByCode(String code); 8 rtapastau

    @Query(value = "SELECT l FROM Language l WHERE (lower(l.isoCode) = lower(?!))") 3 usages 8 rtapastau
    Optional<Language> findByIsoCode(String code);

    @Query(value = "SELECT l FROM Language l WHERE code in (?!)*") 3 usages 8 rtapastau
    List<Language> findByListCode(List<String> code);
}
```

Figura 30: Classe *LanguageRepository*. Font: Elaboració pròpia.

Diagrama de seqüència d'editar les aplicacions d'un usuari

Per a un millor enteniment de com interaccionen les capes i els seus elements mencionats anteriorment, es mostrarà un exemple per a un cas real. Utilitzaré el cas d'ús a implementar

d'assignar o revocar accés a les aplicacions del portal als usuaris, més concretament, quan un administrador edita el llistat d'aplicacions ja en la interfície de gestió d'aplicacions dels usuaris.

Figura 31: Interfície d'editar aplicacions d'usuari. Font: Elaboració pròpia.

Una vegada l'administrador modifica la informació i prem el botó de *Save*, el diagrama de seqüència que se segueix és el següent:

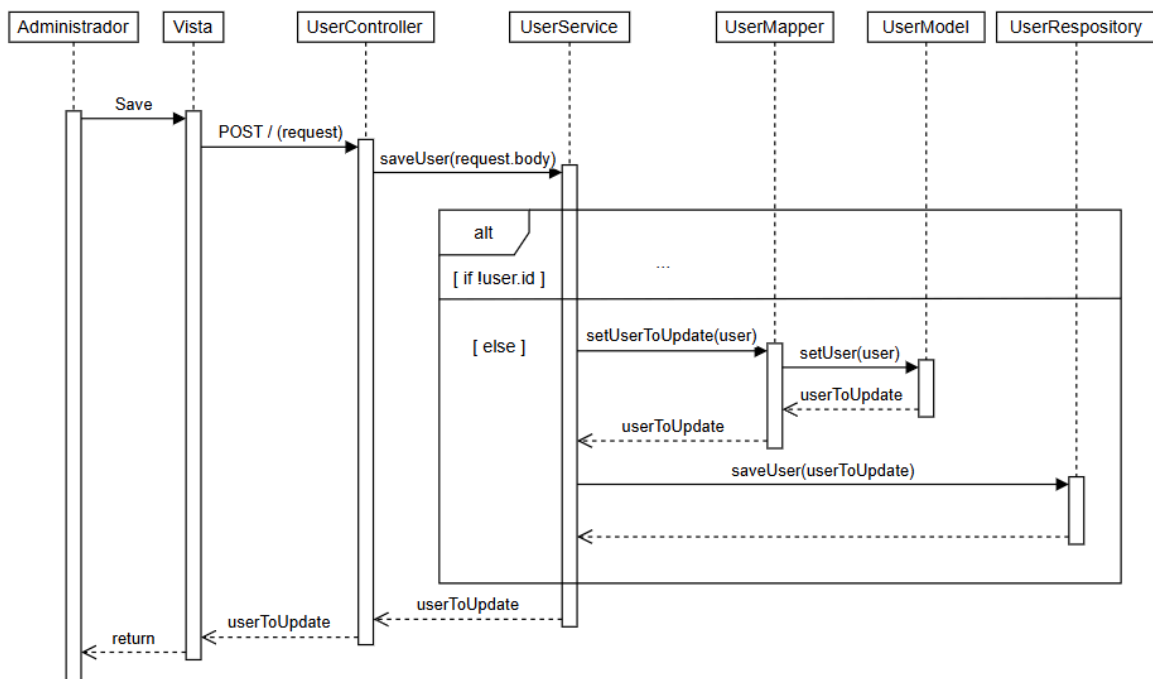


Figura 32: Diagrama de seqüència d'edició d'aplicacions d'usuari. Font: Elaboració pròpia.

6. Implementació

En aquest apartat es llisten les tecnologies i eines de desenvolupament emprades en aquest projecte, algunes ja mencionades en altres apartats, que ens permeten desenvolupar aplicacions web modernes i escalables.

6.1. Tecnologies emprades

Angular 11+

Angular és un *framework* de desenvolupament web creat per Google, basat en el llenguatge de programació superconjunt de JavaScript anomenat TypeScript [\[23\]](#), que permet construir aplicacions web modernes d'una sola pàgina (SPA). Permet crear interfícies d'usuari dinàmiques, interactives i escalables, facilitant el desenvolupament i el manteniment de grans aplicacions. Aquesta tecnologia s'utilitza pel *frontEnd* de l'aplicació d'aquest projecte.

Spring Data & JPA 2.2

Spring Data és un projecte de Spring [\[24\]](#) que simplifica l'accés i gestió de dades en aplicacions Java [\[25\]](#), mentre que JPA (Java Persistence API) és una especificació que defineix com mapar objectes Java a bases de dades relacionals. Aquesta combinació s'utilitza per gestionar la persistència de dades, cosa que permet interactuar amb bases de dades d'una manera més senzilla i orientada a objectes. Entre els principals avantatges destaquen la reducció de la quantitat de codi necessari per accedir a les dades, la facilitat per canviar de base de dades amb menys esforç. Aquesta tecnologia ens ajuda en la comunicació entre la capa de dades i la base de dades de l'aplicació del projecte. Tant aquesta tecnologia com les següents estan pensades per a ser utilitzades en desenvolupament d'aplicacions Java, el llenguatge que usarem al llarg del projecte.

Spring Boot 2.2.2

Spring Boot és un *framework* que simplifica la creació d'aplicacions Java basades en Spring, oferint configuració automàtica i eines que permeten desplegar aplicacions de manera ràpida. Es fa servir per accelerar el desenvolupament de microserveis i aplicacions empresarials, reduint la complexitat de configuració. Facilita la gestió de les dependències i el desplegament.

Java OpenJDK 17

OpenJDK és una implementació lliure i oberta del Java Development Kit (JDK), la plataforma per desenvolupar i executar aplicacions Java. S'usa com a entorn d'execució i desenvolupament per aplicacions Java, aprofitant les millores de rendiment i seguretat que aporten les versions actuals. A més, garanteix compatibilitat amb les últimes especificacions del llenguatge Java.

A l'hora de prendre la decisió d'utilitzar Angular pel desenvolupament del *frontEnd* i Spring pel *backEnd* es van considerar diversos criteris tecnològics, es va pensar en la metodologia de treball que s'implementaria al llarg del desenvolupament de l'aplicació i es va tenir en consideració la mantenibilitat del projecte a llarg termini. Tant Angular com Spring són *frameworks* madurs i àmpliament provats en entorns de producció, cosa que ofereix una gran estabilitat al projecte. El fet que siguin estandarditzats implica que s'han anat perfeccionant al llarg dels anys, resolent o mitigant moltes de les possibles incidències que podrien sorgir en fases de desenvolupament, integració o manteniment. Per tant, els riscos relacionats amb canvis sobtats, incompatibilitats o disrupcions en les seves versions són molt menors en comparació amb altres tecnologies més emergents.

Ambdós *frameworks* compten amb el suport d'empreses líders del sector tecnològic: Angular és mantingut per Google, mentre que Spring forma part de l'ecosistema de Pivotal/VMware. Aquest suport institucional garanteix un suport tècnic de qualitat, la compatibilitat amb els estàndards empresarials més exigents i la disponibilitat regular d'actualitzacions i millores de seguretat. Disposar d'una gran varietat de serveis integrats i eines oficials facilita la integració amb altres solucions, així com el desplegament i la gestió eficient dels sistemes.

Per un altre costat, aquestes tecnologies compten amb una gran comunitat activa d'usuaris i desenvolupadors que generen constantment recursos didàctics, documentació exhaustiva, tutorials, llibreries addicionals i eines de manteniment agilitzant el procés d'aprenentatge així com la resolució d'incidències. També és més senzill trobar professionals qualificats per aquestes tecnologies, cosa que redueix el risc que el projecte quedi obsolet per manca de manteniment en el futur i garanteix la seva escalabilitat i adaptabilitat.

Finalment, cal mencionar que la utilització de *frameworks* consolidats facilita la gestió del projecte durant tot el seu cicle de vida. Les bones pràctiques, les metodologies d'implementació recomanades i la compatibilitat amb eines de test, integració contínua i

desplegament automàtic són factors fonamentals per garantir la maduresa, l'eficiència i la sostenibilitat del projecte a llarg termini.

6.2. Eines de desenvolupament

6.2.1. Repositoris

En aquest projecte es té un repositori per a cada microservei i client, exceptuant el *Leader* el qual no està duplicat. A més, els *frontEnds* de l'aplicació Commercial Cloud també tenen un repositori propi per a cada client anomenat *Adminapp*. En total, tenim amb aquests 11 repositoris:

- cc-leader
- cc-seat-portal
- cc-vged-portal
- cc-seat-adminapp
- cc-vged-adminapp
- cc-seat-access-control
- cc-vged-access-control
- cc-seat-scheduler
- cc-vged-scheduler
- cc-seat-communication-log
- cc-vged-communication-log

Els repositoris que més es tocan en aquest projecte són els de *Access Control*, *Adminapp*, *Leader* i *Portal*.

La gestió de cada repositori es fa mitjançant Gitflow [\[26\]](#). Aquesta metodologia defineix un conjunt clar de branques amb propòsits específics i un flux de treball per gestionar desenvolupament, proves i llançament de l'aplicació.

En cada repositori es defineixen tres branques principals:

- DEV: És la branca on es desenvolupen noves funcionalitats i s'integren els canvis inicials.
- QA: Aquesta branca s'utilitza principalment per proves. Només s'incorporen canvis quan la funcionalitat a DEV ja està acabada i testejada pel desenvolupador.
- PROD: És la branca de producció on només es fusionen desenvolupaments testejats i aprovats, llestos per ser ensenyats als usuaris.

El flux de treball comença en el desenvolupament de noves funcionalitats. Cada nova funcionalitat es desenvolupa en una branca derivada de la de DEV (*feature*). Una vegada acabada i testejada pel mateix desenvolupador, es fusiona la branca a DEV.

A continuació, per integra a QA i fer una ronda de proves més formal es fa un *merge* de la branca DEV cap a la de QA. S'executen les proves corresponents i, si es troba un error, es soluciona creant una branca *hotfix* sobre QA o tornant a DEV depenent de la gravetat de l'error.

Per acabar, quan QA és estable i les proves són aprovades, es fusiona a PROD. Aquest *merge* dona lloc a la versió que veuran els usuaris/clientes. Si es detecta un error, també es pot crear una branca *hotfix* sobre PROD.

És important per al bon funcionament d'aquesta metodologia fusionar tots canvis realitzats a una branca a la resta de branques per mantenir-les sincronitzades.

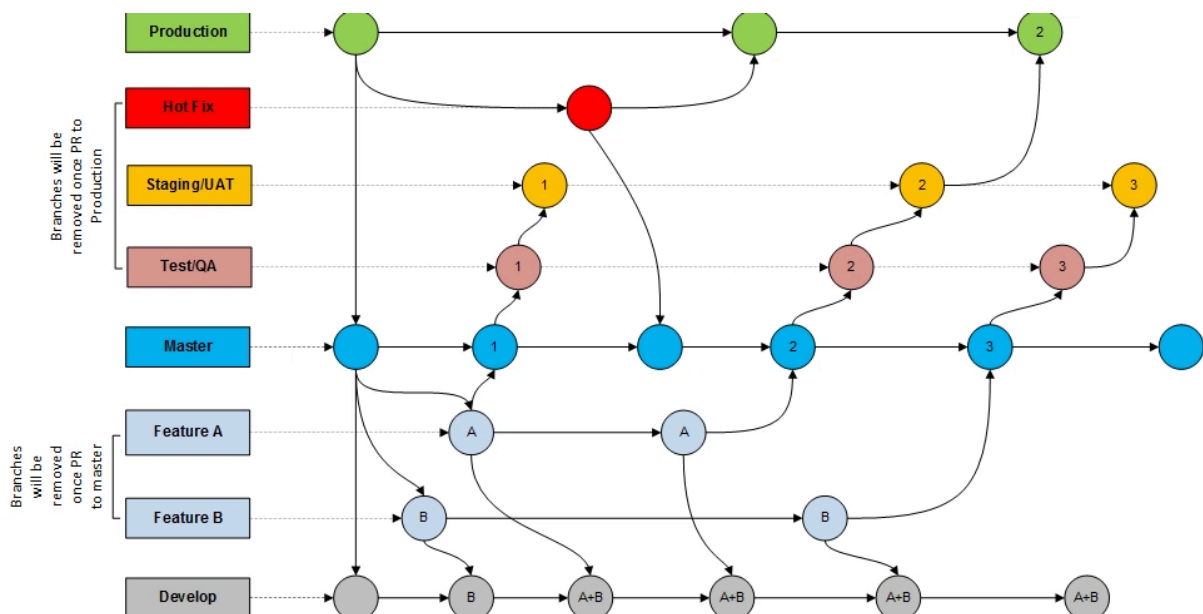


Figura 33: Gitflow. Font: <https://dev.to>.

6.2.2. IDE

Els dos IDE (Entorn de Desenvolupament Integrat) utilitzats en aquest projecte han sigut IntelliJ IDEA i Visual Studio Code (VS Code).

IntelliJ IDEA

IntelliJ IDEA és un entorn de desenvolupament integrat (IDE) creat per JetBrains, pensat principalment per al desenvolupament de programari en Java, tot i que ofereix compatibilitat amb altres llenguatges.. Aquesta eina facilita el procés de desenvolupament

d'aplicacions proporcionant funcions avançades d'edició de codi, depuració, refactorització, gestió de projectes, integració amb sistemes de control de versions i una àmplia selecció de *plugins*. En aquesta eina es desenvolupa tota la part de *backEnd* de l'aplicació, és a dir, la capa d'aplicació i de dades.

Visual Studio Code (VS Code)

Visual Studio Code és un editor de codi font lleuger, multiplataforma i gratuït desenvolupat per Microsoft, que permet treballar amb una gran varietat de llenguatges de programació gràcies a la seva arquitectura d'extensions. Es fa servir per editar codi de manera eficient i àgil, oferint funcionalitats com la gestió de projectes, la integració amb sistemes de control de versions, la depuració i una gran gamma d'extensibilitat adaptables a cada desenvolupador. En aquesta eina es desenvoluparà tot el *frontEnd* de l'aplicació, la capa de presentació.

7. Desplegament

Pel desplegament de l'aplicació s'ha fet ús d'Amazon EKS, un servei que ofereix AWS el qual permet utilitzar Kubernetes (K8s), una plataforma d'orquestració de contenidors creada per Google, sense necessitat d'instal·lar-la ni mantenir-la. Permet automatitzar el desplegament, escalat i gestió d'aplicacions que corren en contenidors, executar milers de contenidors en diferents servidors i assegurar alta disponibilitat.

Respecte a l'organització dels recursos i components d'Amazon EKS per a la nostra aplicació, cada microservei es desplegarà en un *pod*, la unitat més petita de desplegament en Kubernetes. Com que tenim una aplicació per client, cada client tindrà un *namespace* [27] concret que agrupa els microserveis de l'aplicació. És a dir, els microserveis estaran duplicats per a cada *namespace*. A part, hi haurà un *namespace* comú pel microservei del *Leader*, el qual no duplicarem, ja que és comú per tot microservei. Per acabar, per a cada entorn de desenvolupament que tenim (DEV, QA i PROD) s'utilitzarà un clúster diferent. En resum, l'organització serà la següent:

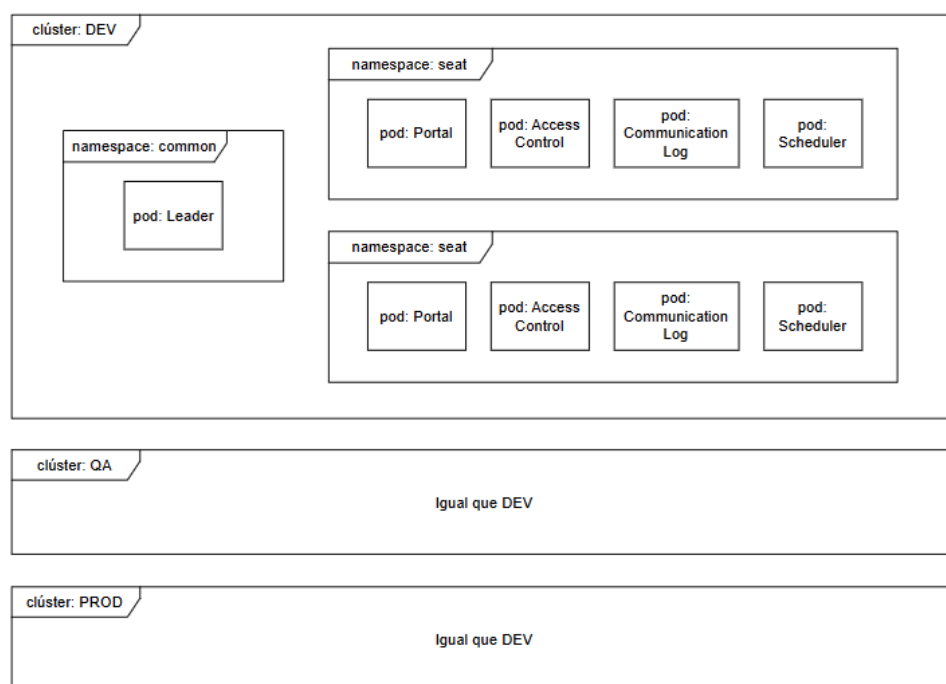


Figura 34: Organització del desplegament. Font: Elaboració pròpia.

El procés de desplegament per la nostra aplicació comença per la creació d'una imatge Docker [28] per a cada microservei el qual té el seu propi repositori amb un Dockerfile [29]. Aquestes imatges es guarden a l'Amazon ECR (Elastic Container Registry). A continuació,

s'han de configurar els manifests que és on s'indica quina imatge Docker utilitzar, quants *pods* córrer, els ports a exposar i detalls de configuració entre altres. Finalment, s'executa un *deploy* indicant el microservei a desplegar i en quin *namespace* i entorn desplegar-lo.

AWS és una plataforma *cloud* consolidada amb serveis robustos i una infraestructura fiable. Un dels grans avantatges d'AWS és que disposa d'una selecció molt àmplia de serveis: des del desplegament de servidors i bases de dades, fins a IA, emmagatzematge, seguretat i serveis específics per sectors. Aquesta diversitat fa que sigui fàcil adaptar-se a les necessitats del projecte, permet créixer i escalar el projecte de manera àgil i elimina la necessitat de buscar altres plataformes quan els requisits evolucionen. Per això s'ha triat AWS en comptes d'altres opcions com Google Cloud Platform o Microsoft Azure. El fet de disposar de tants serveis disponibles deixa centrar l'esforç en el desenvolupament de la solució, sense haver d'invertir temps extra en qüestions d'infraestructura. Una altra raó per la qual es va triar AWS va ser que aquesta plataforma inverteix molt en seguretat i certificacions, gestiona múltiples nivells de seguretat i ofereix eines per protegir dades i infraestructures facilitant el compliment de normatives legals i estàndards internacionals.

8. Validació

Durant el desenvolupament de les funcionalitats s'han dut a terme diverses proves per a garantir el correcte funcionament d'aquestes. Per un costat, s'han dut a terme proves unitàries i d'integració per a cada microservei, i per l'altre, s'han executat tests de rendiment de les consultes refactoritzades i diferents tests de qualitat de l'aplicació.

8.1. Tests unitaris

Per a les proves unitàries s'ha fet ús de les llibreries JUnit 5 [30] i Mockito [31]. La primera és el motor de testatge més utilitzat per Java i s'encarrega d'executar mètodes de test, comprovar resultats i mostrar errors. El segon, com es pot veure en el mateix nom, és una llibreria de *mocks* (simulacions) la qual permet provar una classe sense dependre d'altres components reals.

Amb aquestes proves s'ha cobert cada cas d'ús de cada funció per a totes les classes de cada microservei. A la següent figura es mostren els tests escrits per a la funció d'obtenir la llista d'aplicacions del portal.

```
@Test
@DisplayName("getUsersApplications returns paginated UserApplicationsDTO list")
void testGetUsersApplications_Success() throws DefaultAccessControlException {
    Application application = new Application();
    application.setId(1L);
    UserApplicationsDTO userApplicationsDTO = new UserApplicationsDTO();
    userApplicationsDTO.setApplications(List.of(application));

    List<UserApplicationsDTO> resultList = List.of(userApplicationsDTO);

    when(entityManager.createQuery(anyString(), eq(UserApplicationsDTO.class))).thenReturn(typedQueryUserApplication);
    when(typedQueryUserApplication.getResultList()).thenReturn(resultList);
    when(typedQueryUserApplication.setFirstResult(anyInt())).thenReturn(typedQueryUserApplication);
    when(typedQueryUserApplication.setMaxResults(anyInt())).thenReturn(typedQueryUserApplication);

    // Act
    PageImpl<UserApplicationsDTO> result = userService.getUsersApplications( id: 1L, code: null, application: null, page: 0, pageSize: 10);

    // Assert
    assertNotNull(result);
    assertEquals(resultList.size(), result.getContent().size());
    verify(entityManager).createQuery(anyString(), eq(UserApplicationsDTO.class));
    verify(typedQueryUserApplication).getResultList();
}

@Test
@DisplayName("getUsersApplications throws DefaultAccessControlException on error")
void testGetUsersApplications_Exception() {
    when(entityManager.createQuery(anyString(), eq(UserApplicationsDTO.class))).thenThrow(new RuntimeException("DB error"));

    assertThrows(DefaultAccessControlException.class, () ->
        userService.getUsersApplications( id: 1L, code: "code", application: "app", page: 0, pageSize: 2)
    );
}
```

Figura 35: Exemple de proves unitàries. Font: Elaboració pròpia.

A més, per a garantir una bona qualitat de codi i testatge, una vegada passats tots els tests unitaris, s'ha fet una anàlisi de cada microservei mitjançant SonarQube. SonarQube és una aplicació web que analitza estàticament el codi i ofereix recomanacions i estadístiques dels resultats. En els resultats també es detecten *bugs*, vulnerabilitats de seguretat, problemes de mantenibilitat, la quantitat de codi cobert en els tests, duplicació de codi i *code smells*. Els microserveis on s'ha fet servir el SonarQube són el *Access Control*, *Communication Log* i *Scheduler*, els quals tenen lògica de negoci complexa.

Els resultats finals de les anàlisi han sigut els que es mostren en les figures 30 i 31. Com es pot observar, cap microservei té vulnerabilitats ni *bugs* i tenen un nombre mínim de *code smells*. També s'observa que tots els microserveis tenen un alt percentatge de codi cobert pels tests unitaris i un percentatge negligible de codi duplicat. En general, tots els microserveis passen els estàndards de qualitat de SonarQube.

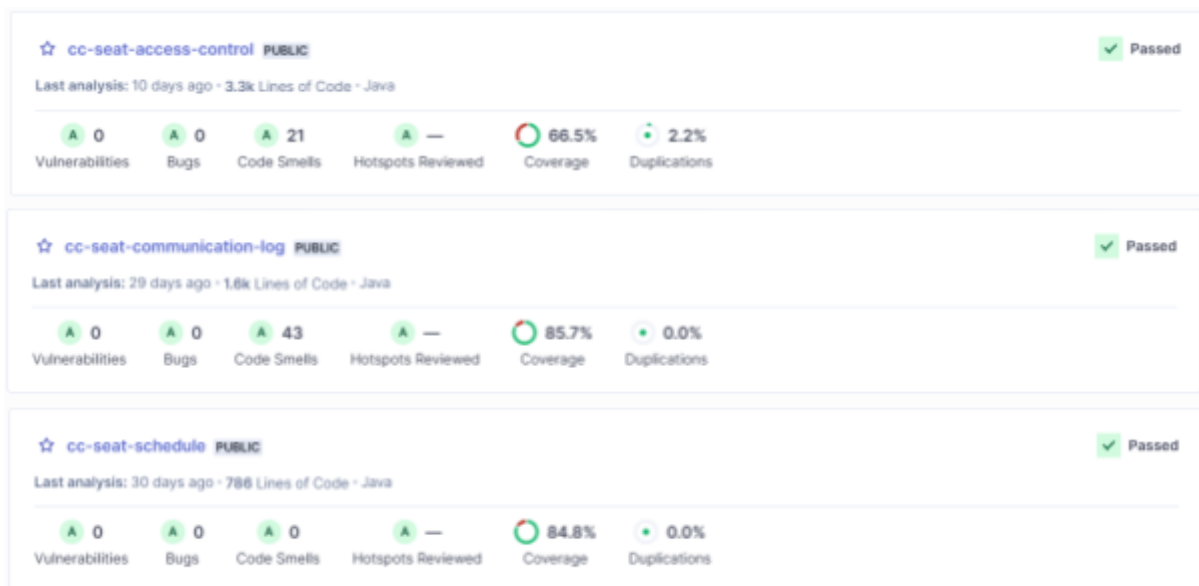


Figura 36: Resultats de SonarQube pels microserveis de SEAT. Font: Elaboració pròpia.

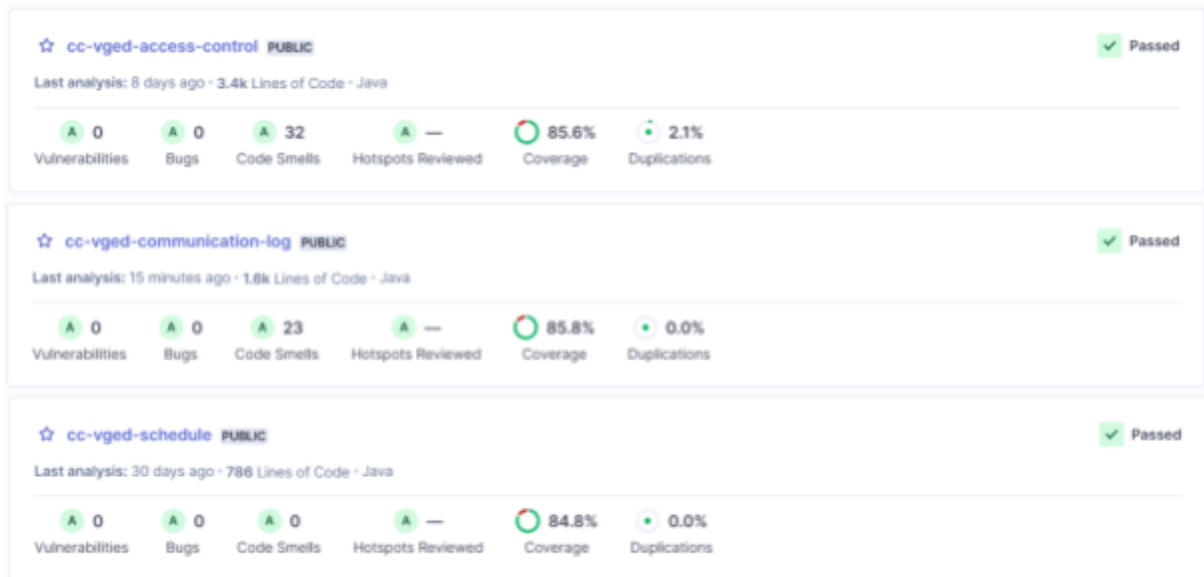


Figura 37: Resultats de SonarQube pels microserveis de VGED. Font: Elaboració pròpia.

Per arribar a aquests resultats ens hem ajudat de la funcionalitat de SonarQube de classificar els problemes segons la seva gravetat i, per a cada problema, explicar per què pot ser un problema i com es pot resoldre. S'han resolt tots els problemes amb gravetat *Major* o superior. Dels problemes més comuns que es troben són mètodes massa llargs o complexos, blocs de codi comentat i variables no utilitzades.

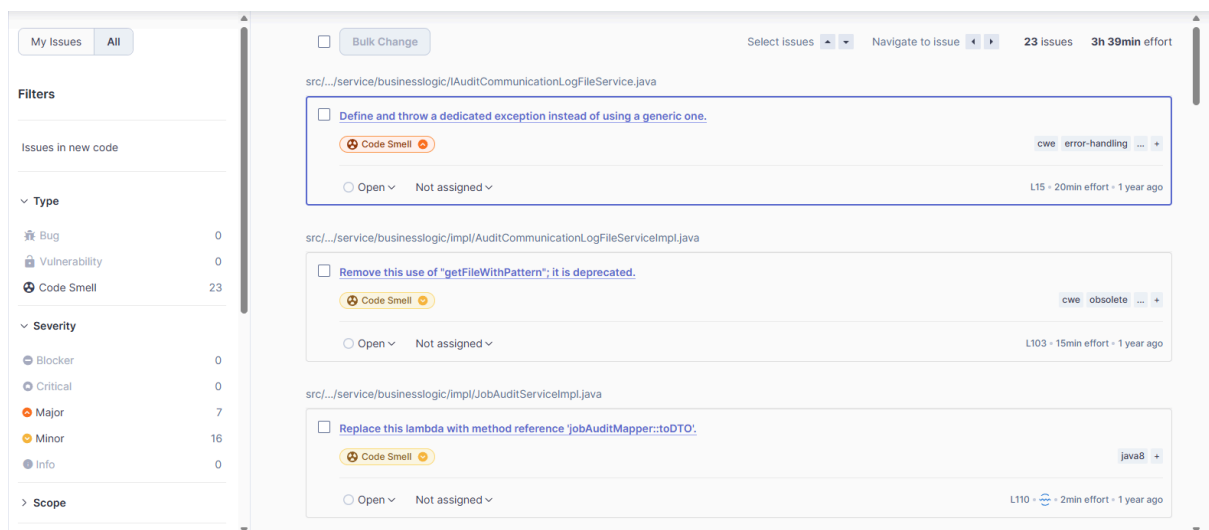


Figura 38: Exemple de classificació de problemes a SonarQube. Font: Elaboració pròpia.

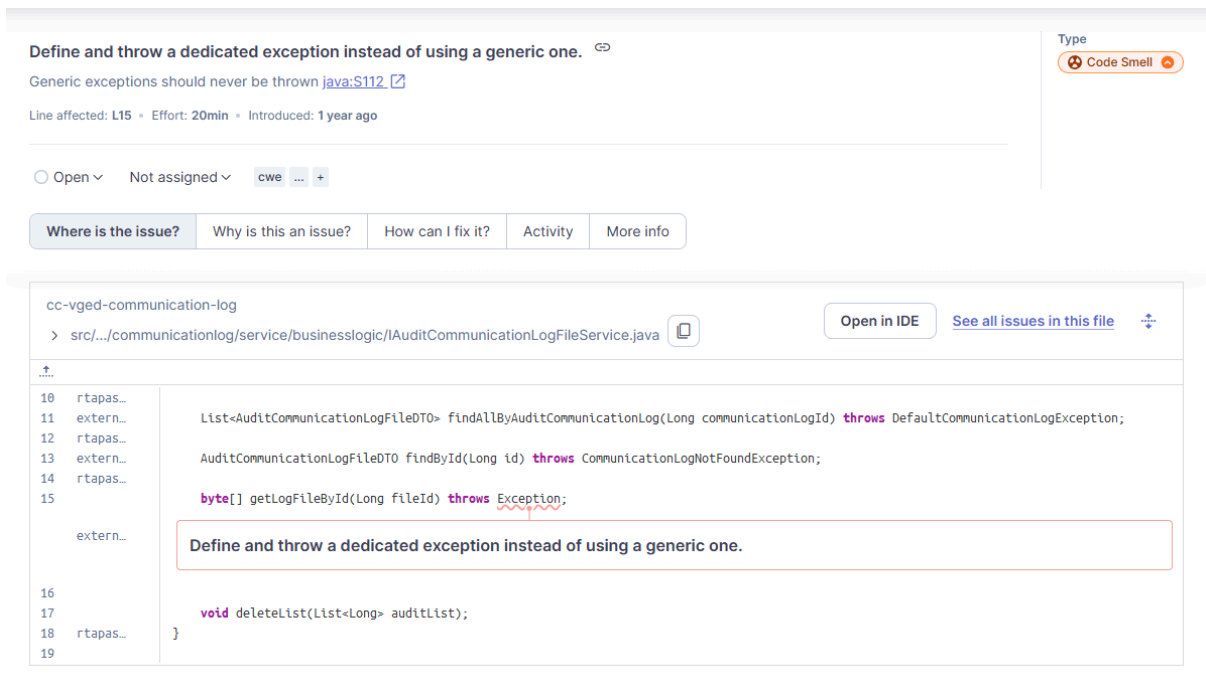


Figura 39: Exemple d'especificació d'un problema a SonarQube. Font: Elaboració pròpia.

8.2. Tests d'integració

Les proves d'integració s'han executat per validar totes les comunicacions entre els diferents microserveis, amb especial atenció als microserveis del *Leader*, *Access Control* i el del portal, ja que són els implicats en les noves funcionalitats implementades. Així mateix, s'han verificat les interaccions entre el *frontEnd* i el *backEnd* de l'aplicació.

Aquestes proves primer s'han dut a terme mitjançant l'arrencada de l'aplicació en un entorn local, on s'han provat de manera exhaustiva totes les casuístiques possibles per garantir la correcta integració i comunicació entre els microserveis. Aquestes casuístiques s'han extret dels casos d'ús especificats anteriorment i s'han llistat en un full d'Excel per registrar l'estat de cada prova i, en cas de trobar-ne, gestionar la resolució de defectes.

Una vegada l'aplicació puja a un nou entorn, es tornen a fer les mateixes proves per verificar que no s'ha espatllat res durant la pujada. A més, per a les tasques del portal i les de la separació per importador s'han documentat evidències amb captures de pantalles per a demostrar el correcte desenvolupament de les tasques.

Per a la tasca **[RM1] Modificar consultes a GSB des de Ordering** s'han dut a terme uns tests d'integració diferents de la resta. Per aquesta tasca s'ha fet ús de l'aplicació Postman [32] per a poder provar les crides modificades donat que no hi ha pantalles del microservei Ordering a l'aplicació. Tant les crides com els JSON per a les peticions les vam

obtenir d'una col·lecció Postman de l'equip que desenvolupa aquest microservei. S'arrenca el microservei en local, es fan les crides a testejar amb el JSON corresponent i es verifica que tant la resposta sigui un OK com que el contingut de la resposta correspongui amb la crida.

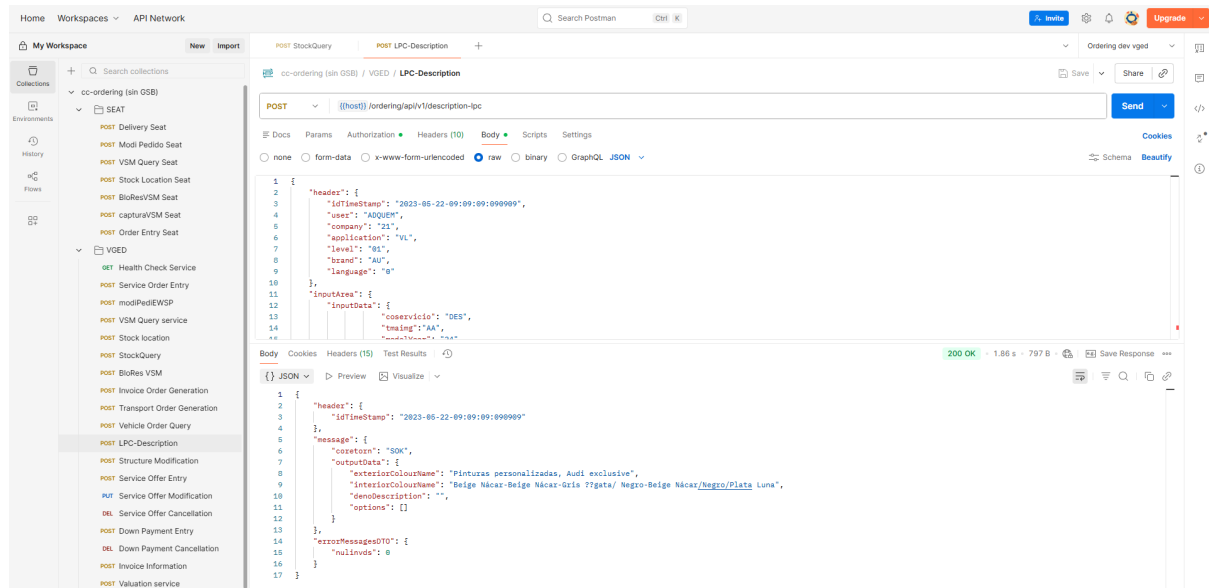


Figura 40: Crida amb Postman d'una operació de Ordering. Font: Elaboració pròpia.

8.3. Tests de rendiment

Aquestes proves només s'han dut a terme per a les classes de les *queries* refactoritzades, és a dir, pels *Users*, *Clusters*, *Roles* i *Authorities*. A priori, s'ha assegurat l'existència de suficient volum de dades per obtenir resultats significatius creant objectes proves segons la necessitat. Després, s'ha navegat per les pantalles de les classes corresponents efectuant diverses operacions de consulta, creació i edició de dades per a verificar que el rendiment de les consultes ha millorat sense afectar el de les altres. Per a més informació, també s'ha estressat el sistema efectuant moltes consultes seguides per a veure com reaccionava.

En aquestes proves s'ha obtingut gràfics amb l'ajut de Grafana [33]. Spring Boot té suport nadiu per exposar mètriques i informació de l'estat de l'aplicació a través del mòdul Spring Boot Actuator [34] que després poden ser visualitzades a Grafana.

La informació en la qual teníem interès en aquest cas era l'ús de CPU pel sistema i el procés, el gràfic de baix a l'esquerra de la figura a continuació, i la mitjana de càrrega del sistema, el gràfic de baix a l'esquerra de la figura a continuació. La resta d'informació que mostra el Grafana és el temps que porta activa la instància analitzada, el percentatge de memòria utilitzada pels objectes en execució, el percentatge de memòria per classes, compilacions, etc. i el nombre d'arxius oberts pel procés.

A la figura a continuació es pot veure els resultats d'anàlisi en realitzar 100 peticions seguides de cerca de tots els permisos de l'aplicació. Es va observar que l'ús màxim de CPU pel procés era d'aproximadament 2,7% i que la mitjana de càrrega del sistema es mantenia al voltant de 8%. A més, el temps d'execució mitjana per a la petició era d'uns 150 mil·lisegons. Aquests números tan baixos demostren que l'aplicació compleix amb els requisits de rendiment establerts.

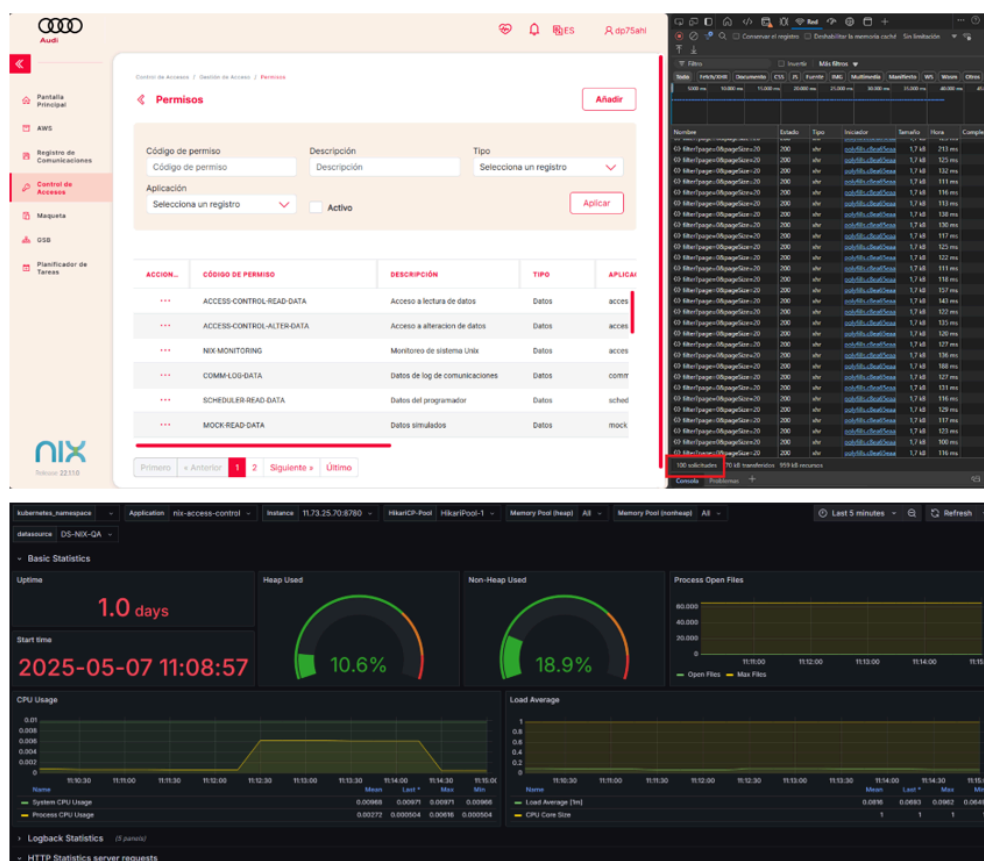


Figura 41: Exemple de resultat d'una prova de rendiment. Font: Elaboració pròpia.

En la següent taula es mostren els resultats de tots els tests de rendiment realitzats. Per les peticions de creació i edició d'un element només s'observava el temps d'execució, ja que no s'han modificat aquestes operacions. Com és d'esperar, els elements amb més informació són els que tenen percentatges i temps més alts.

Petició	Ús de CPU (%)	Càrrega del sistema (%)	Temps d'execució (ms)
Users			
Cerca (1 vegada)	0,01	3	138
Cerca (100 vegades)	1	17	135
Creació	-	-	412
Edició	-	-	614

Clusters			
Cerca (1 vegada)	0,02	7	115
Cerca (100 vegades)	2,2	10	115
Creació	-	-	651
Edició	-	-	542
Roles			
Cerca (1 vegada)	0,007	5	148
Cerca (100 vegades)	0,3	12	135
Creació	-	-	453
Edició	-	-	106
Authorities			
Cerca (1 vegada)	0,01	1	154
Cerca (100 vegades)	2,7	8	150
Creació	-	-	347
Edició	-	-	356

Taula 33: Resultats de tests de rendiment. Font: Elaboració pròpia.

8.4. Tests de qualitat

Una vegada una gran part de les tasques del projecte estaven completades, s'han dut a terme diferents proves de qualitat per a verificar el compliment dels requisits no funcionals.

Per un costat, per a provar tots els aspectes relacionats amb la usabilitat de l'aplicació s'ha demanat a diferents persones amb cap coneixement de l'aplicació que naveguin per ella durant un parell d'hores i informin dels problemes amb els quals s'han trobat. Pels suggeriments i comentaris que hem rebut, i que aplicaven dins del context de l'aplicació, s'han fet les modificacions adients.

Per una altra banda, per a millorar l'estètica de l'aplicació també s'ha donat accés a l'aplicació a un dissenyador d'interfícies el qual ha detectat varies petites millores a aplicar. Aquesta persona sí que té coneixements sobre el client de l'aplicació i no només va analitzar l'organització i l'estil dels components de la interfície, sinó que també ha pogut detectar millores com la necessitat d'actualitzar els colors i la tipografia que s'utilitzava en l'aplicació.

En les següents dues imatges es poden identificar alguns canvis de la interfície resultants del test. Específicament, es mostren els canvis per a la pantalla del microservei *Access Control*,

on es poden veure la majoria dels canvis. El primer que podem notar és que el color vermell ha sigut substituït per un blau grisós i les icones dels botons han sigut substituïts per uns altres amb colors més neutres perquè mantinguin l'estètica en totes les marques. També es pot veure que tot i haver afegit un nou botó “User Applications”, el disseny de l'aplicació es manté uniforme. Una altra modificació és el canvi de logotips tant el de la marca Audi a la part superior esquerra com el de l'aplicació a la part inferior esquerra de la pantalla. Altres modificacions observables són els espaiats entre botons i la tipografia dels textos de l'aplicació.

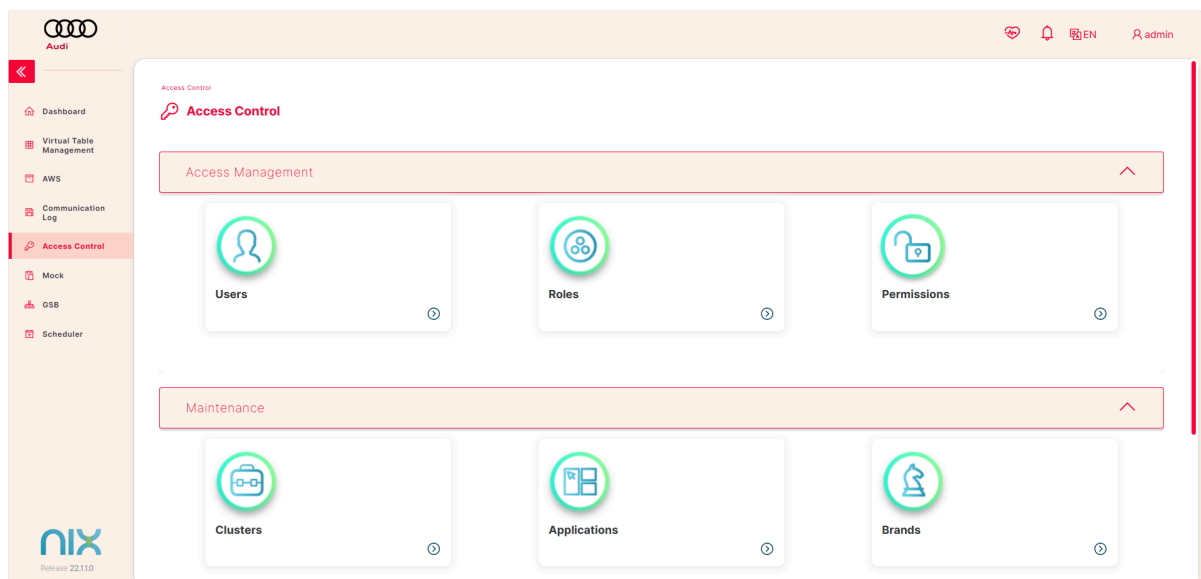


Figura 42: Pantalla antiga d'Access Control per Audi. Font: Elaboració pròpia.

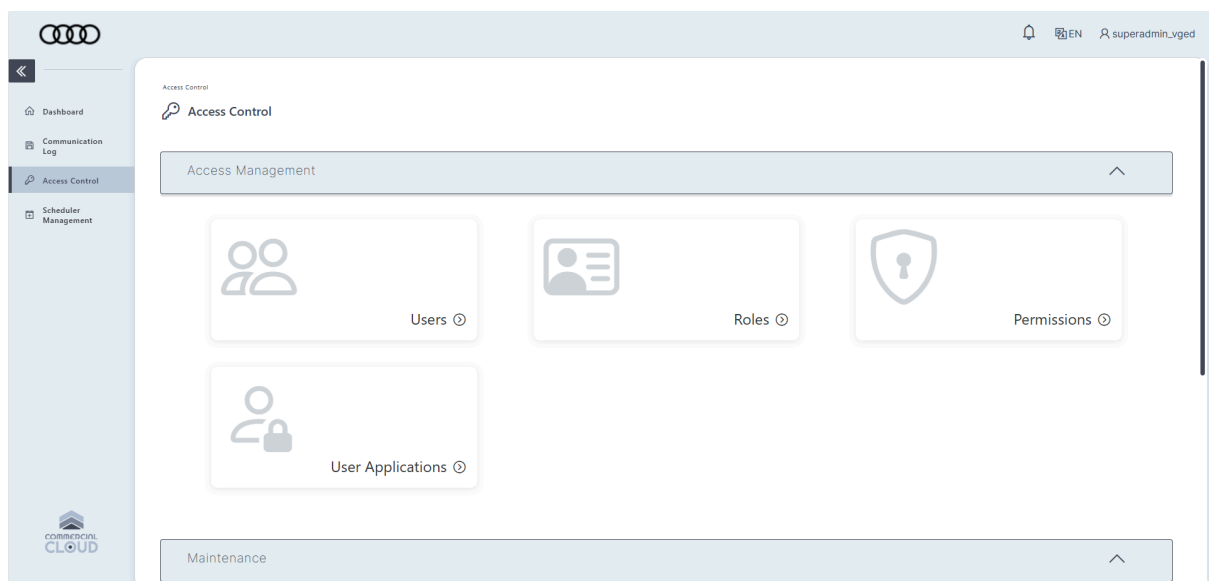


Figura 43: Pantalla nova d'Access Control per Audi. Font: Elaboració pròpia.